

生成 AI 時代のソフトウェア工学

森 直樹

大阪公立大学 情報学研究科

目次

序章 本書の使い方	ix
第 I 部 ソフトウェア工学の基礎	1
第 1 章 ソフトウェア工学の基礎	3
1.1 ソフトウェアの来歴と工学化	3
1.1.1 ソフトウェアが社会基盤になるまで	3
1.1.2 ソフトウェア危機とソフトウェア工学の誕生	4
1.1.3 ソフトウェア工学とは何か	4
演習	5
第 2 章 品質, 信頼性, 安全性	7
2.1 本章の位置づけ	7
2.2 品質モデル	7
2.3 信頼性, 可用性, 安全性	9
2.4 測定とその限界	10
2.5 事故から学ぶ	10
2.6 品質目標と障害時方針	10
2.7 専門職倫理	11
演習	11
第 3 章 要求工学	13
3.1 本章の位置づけ	13
3.2 要求の種類	13
3.3 良い要求文	14
3.4 ユースケースと受入基準	14
3.5 研究室備品予約システムの最小導入	15
研究室備品予約システムの最小導入	15
3.6 要求一覧の初稿	15
3.7 要求から設計へ	16
演習	16
第 4 章 モデリングと設計	17
4.1 本章の位置づけ	17
4.2 モデルは目的に応じて現実を削る	18
4.3 オブジェクト指向の基本	18
4.4 UML 主要図	19

4.4.1	多重度を読む	19
4.5	設計原則とパターン	19
4.6	並行性モデル: ペトリネット入門	20
4.6.1	なぜペトリネットを学ぶか	20
4.6.2	四つの要素	20
4.6.3	発火規則	21
4.6.4	排他制御の例: 備品の取り合い	21
4.6.5	デッドロックの直観	21
4.6.6	現代での位置づけ	22
	演習	22
第 5 章	Python プログラム読解	23
5.1	本章の位置づけ	23
5.2	名前とオブジェクト	23
5.3	可変性と共有渡し	24
5.4	浅いコピーと深いコピー	24
5.5	型ヒントと実行時検査を分ける	25
5.6	例外と失敗の表現	25
5.7	非同期処理の入口	26
	演習	26
第 6 章	データモデリングとデータベース	27
6.1	本章の位置づけ	27
6.2	データ独立性	27
6.3	ER モデル	27
6.4	関係モデルと関係代数	28
6.5	SQL の集計	28
6.6	キーと制約	29
6.7	正規化	29
6.8	トランザクションと ACID	30
6.9	スキーマは変更される	30
	演習	30
第 7 章	テスト, レビュー, 品質保証	31
7.1	本章の位置づけ	31
7.2	テストレベル	31
7.3	ブラックボックステスト	31
7.4	ホワイトボックステスト	32
7.5	TDD	32
7.6	レビュー技法	33

7.7	証拠を組み合わせる	33
7.8	研究室備品予約システムの検証計画	33
	研究室備品予約システムの検証計画	33
	演習	34
第 8 章	開発プロセス, アジャイル, DevOps	35
8.1	本章の位置づけ	35
8.2	工程モデルの歴史	35
8.3	アジャイル諸派	36
8.4	スクラム	36
8.5	XP のプラクティス	37
8.6	DevOps, DORA, SRE	37
8.7	完了条件と検査	37
	演習	38
第 II 部	AI 駆動開発への応用	39
第 9 章	大規模言語モデルの基礎	41
9.1	本章の位置づけ	41
9.2	言語モデルとトークン	41
9.3	生成 AI の系譜: NLP から LLM へ	42
9.4	サンプリング	42
9.5	学習段階	43
9.6	ハルシネーション	43
9.7	アプリとモデルを分ける	44
9.8	計算資源と環境制約	44
	演習	46
第 10 章	プロンプト設計と LLM 評価	47
10.1	本章の位置づけ	47
10.2	プロンプトは仕様である	47
10.3	出力形式と禁止事項	48
10.4	小さな評価セットを作る	48
10.5	LLM 応答の評価	49
10.6	モデルとプロンプトの比較	49
10.7	採用判断を記録する	50
	演習	50
第 11 章	RAG と知識接続	51
11.1	本章の位置づけ	51
11.2	検索基盤	52
11.2.1	埋め込み	52

11.2.2	近似最近傍探索	52
11.2.3	チャンキング	52
11.3	RAG アーキテクチャ	53
11.3.1	基本パイプライン	53
11.3.2	ハイブリッド検索とリランキング	53
11.3.3	RAG 評価	53
11.4	安全性と運用	54
11.4.1	プロンプトインジェクション	54
11.4.2	権限と監査	54
11.4.3	研究室備品予約システムの規約 RAG	54
	研究室備品予約システムの規約 RAG	54
	演習	54
第 12 章	AI エージェント	55
12.1	本章の位置づけ	55
12.2	LLM API と信頼性設計	55
12.3	状態, 計画, ツール	56
12.4	ツール権限	56
12.5	推論と行動のパターン	57
12.6	Human-in-the-Loop	57
12.7	監査ログとキルスイッチ	57
12.8	研究室備品予約システムの予約調整エージェント	57
	研究室備品予約システムの予約調整エージェント	57
	演習	58
第 13 章	AI 駆動開発の実践	59
13.1	枠組み: AI 駆動開発はプロセスである	59
13.2	品質目標を先に書く	60
13.3	要求工程	60
13.4	設計工程	61
13.5	実装工程	63
13.6	DB 工程	65
13.7	検証工程	66
13.8	AI ペアプログラミングと AI レビューの規律	67
13.9	記録規律	67
	演習	68
第 14 章	AI 運用, 責任, キャリア	69
14.1	本章の位置づけ	69
14.2	AI 利用ポリシー	69

14.3	AI 利用ログ	70
14.4	秘密情報と禁止入力	70
14.5	モデルカード, データシート, 規制	71
14.6	著作権とライセンス	71
14.7	AIOps と運用責任	71
14.8	専門職倫理の実践	72
14.9	これからのソフトウェアエンジニア	72
14.10	本書の総括	73
	演習	73
	章末コラム	73
付録 A	ペトリネット詳細解析	75
A.1	形式的な定義	75
A.2	到達可能性とデッドロック	75
A.3	有界性と不変量	76
A.4	実務での使い方	76
付録 B	他言語のメモリ・参照・所有権	77
B.1	C と C++	77
B.2	Java と C#	77
B.3	JavaScript, Go, Rust	78
B.4	対応表	78
付録 C	UML 主要図以外の図種一覧	79
C.1	構造を表す図	79
C.2	実装と配置を表す図	79
C.3	相互作用を表す図	79
C.4	図を選ぶ基準	80
付録 D	用語集	81
付録 E	Python 基本構文	89
E.1	環境と実行	89
E.2	変数, 条件分岐, 繰返し	89
E.3	関数とデータ構造	90
E.4	文字列, ファイル, 例外	90
E.5	クラス, モジュール, pytest	90
付録 F	画像生成系の概要	93
F.1	生成モデルの考え方	93
F.2	拡散モデル	93
F.3	GAN と VAE	93
F.4	マルチモーダル	94

F.5	動画生成の概観	94
F.6	ソフトウェア工学から見た注意点	94
参考文献		95
索引		99

序章 本書の使い方

本書の立場

本書の立場は単純である。生成 AI でソフトウェアを速く作る話に閉じず、**ソフトウェア工学とは、生成 AI に何を任せるかを決める裁量の言語である**と考える。要求、設計、テスト、レビュー、構成管理、品質、プロセスは、「どこまで任せるか」「何を固定するか」「どの証拠を見るか」を共有可能な言葉にする。

生成 AI は、ソフトウェア工学の基礎語彙をよく知っている。UML、状態機械、ペトリネット、デザインパターン、テスト駆動開発、レビュー技法、正規化、アジャイル、DevOps は、LLM への強い手がかりにもなる。これらの語彙は、AI に意図を伝え、出力を縛り、誤りを見つけるための現在の言語である。

ただし、AI が語彙を知っていることと、正しく使えることは別である。LLM は、Strategy パターンを提案しながら過剰設計を生み、UML 図らしい図を作りながら多重度を誤り、テストらしいコードを書きながら仕様を追認するだけのことがある。人間は用語を覚えるだけでなく、その用語が何を制約し、何を検証可能にするかを読む。

AI 駆動開発の扱い

本書では、AI 駆動開発を余談にしない。ここでいう AI 駆動開発では、AI に丸投げせず、要求、設計、実装、テスト、レビュー、運用の各段階で、人間が裁量を設計しながら AI を使う。AI の出力は最終成果物ではなく候補である。候補は、要求に照らして読み、設計で縛り、テストで壊し、レビューで判断し、ログとして残す。

本書は、第 I 部「ソフトウェア工学の基礎」と第 II 部「AI 駆動開発への応用」から成る。第 I 部では、AI の有無に関わらず成り立つ要求、設計、品質、データ、テスト、プロセスを学ぶ。第 II 部では、その基礎概念を LLM、プロンプト、RAG、エージェント、AI 駆動開発、運用でどう使うかを学ぶ。二部構成の軸は時間でも優劣でもない。基礎を共有し、その基礎を AI 応用へ渡す流れである。

基礎を学ぶ理由

第 I 部の基礎を厚く扱う理由は、AI に渡す裁量と、AI から返ってきた候補を検証する土台を作るためである。ユースケースは目的を伝える言語であり、UML は構造と振る舞いを伝える言語で

あり、ペトリネットは並行性と資源競合を伝える言語である。テストファーストは実装前に期待を固定し、デザインパターンは変更に対する設計意図を伝える。

生成 AI 時代の開発者には、AI なしで何でも手作業する力に加え、AI が出した候補にどの言葉で指示し、どの観点で疑い、どの証拠で採用するかを説明する力が必要になる。その説明に、基礎語彙は今も効く。第 II 部の各章は、第 I 部で整理した概念を参照し、AI 応用での使い道を示す。

本書の使い方

本書は、ソフトウェア工学と生成 AI の関係を基礎から押さえたい読者を対象とする。Python の基本文法を一通り知っていると読みやすいが、第 5 章では、AI 生成コードを読むために必要な参照、可変性、コピー、型ヒント、例外を扱う。Python の初歩に不安がある読者は、第 5 章へ進む前に付録 E を確認するとよい。Python 以外の言語で参照や所有権の考え方を比べたい読者は、付録 B を参照せよ。

演習では、AI の使用を段階的に増やす。序盤では、指定されたプロンプトを実行し、出力を比較する。中盤では、要求、設計、テストを使って AI 出力を制約する。終盤では、RAG やエージェントを、権限、監査、評価、停止条件を持つソフトウェアとして設計する。

本書の目標は、AI をうまく使ったという感想で終わらせないことにある。AI に任せ部分と人間が判断した部分を説明し、その判断をテスト、レビュー、ログ、根拠で支えられるようになることである。

第I部 ソフトウェア工学の基礎

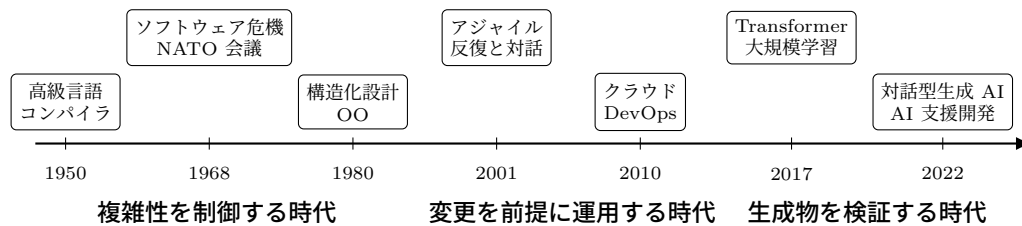


図 1.1 ソフトウェア工学の概略年表。高級言語による複雑性の制御から、アジャイルと DevOps による変更前提の運用へと、主題が移ってきた。

第 1 章 ソフトウェア工学の基礎

ソフトウェアは、いまや社会の基盤である。電力網、金融、物流、医療、行政、教育のいずれも、ソフトウェアなしには 1 日も動かない。その基盤を設計し、作り、動かし続けるための学問がソフトウェア工学 (Software Engineering, SE) である。本章では、ソフトウェアが社会基盤になった流れ、ソフトウェア工学が必要になった理由、そしてソフトウェア工学とは何かを概観する。ここで扱うのは、特定の言語や道具に依らず時間に耐える基礎概念である。

本章の到達目標。

知識 ソフトウェア危機、北大西洋条約機構 (North Atlantic Treaty Organization, NATO) ガーミッシュ会議、本質的複雑性と偶発的複雑性、銀の弾丸といった用語を説明できる。

適用 ソフトウェア工学を「制約下の意思決定」として捉え、具体的な開発状況にあてはめて論じられる。

実践 身近なソフトウェアを、体系的・規則的・定量的という三つの観点から批評できる。

1.1 ソフトウェアの来歴と工学化

1.1.1 ソフトウェアが社会基盤になるまで

ソフトウェアの歴史は、ハードウェアの付属物から社会の基盤へという拡大の歴史である。図 1.1 に、ソフトウェア工学の歩みを大づかみに捉えた概略年表を示す。この年表は本章の地図でもあり、複雑性を制御する時代から、変更を前提に運用する時代へという流れで大づかみに読める。

黎明期の計算機では、計算の手順はハードウェアの配線として固定されていた。プログラム内蔵方式の確立により、手順をデータとして記憶し、書き換えられるようになった。これがソフトウェアという概念の出発点である。その後、FORTRAN や COBOL, LISP のような高級言語が登場

し、人間が読み書きしやすい記法で手順を表せるようになった。UNIX と C 言語は、ソフトウェアを共有し移植する文化を広めた。パーソナルコンピュータの普及は、ソフトウェアを専門家の道具から大衆の道具へと変えた。1990 年代の Web の商用化、2000 年代のクラウドとスマートフォンの普及を経て、ソフトウェアはあらゆる産業の中核に入り込んだ。

この拡大の各段階で、ソフトウェアの規模と複雑さは増し続けた。規模が増せば、1 人の天才が頭の中で全体を把握する作りかたは破綻する。ソフトウェア工学は、この破綻に対する工学的な応答として生まれた。

1.1.2 ソフトウェア危機とソフトウェア工学の誕生

1968 年、ドイツのガーミッシュで開かれた NATO の会議で、ソフトウェア工学という言葉が広く用いられるようになった [NR69]。この会議は、大規模なソフトウェア開発が予算を超過し、納期に遅れ、品質を欠き、ときに完成すらしないという事態を **ソフトウェア危機** (software crisis) として共有した。ハードウェアの性能が急速に伸びる一方で、それを使いこなすソフトウェアを作る能力が追いつかない、という認識である。

この危機の本質を鋭く論じたのが ブルックス (Frederick P. Brooks) である。彼は大規模システムの開発経験から、遅れているプロジェクトに人員を追加するとかえって遅れる、という反直観的な法則を示した [Bro75]。人を増やせば、教育とコミュニケーションの費用も増える。その費用が追加の労力を上回ることがある。

さらにブルックスは、ソフトウェア開発の困難を二種類に分けた [Bro87]。一つは **本質的複雑性** (essential complexity) であり、解こうとしている問題そのものに由来する、取り除けない複雑さである。もう一つは **偶有的複雑性** (accidental complexity) であり、道具や手法の未熟さに由来する、改善で減らせる複雑さである。高級言語や統合開発環境は偶有的複雑性を減らしてきたが、本質的複雑性は残る。だから、一つの技術で生産性を一気に何倍にもする **銀の弾丸** は存在しない、というのが彼の主張である。

本質的複雑性と偶有的複雑性という区別は、本書を通じて繰り返し参照する道具である。どの工程でも、「これは問題そのものの難しさか、それとも道具の未熟さか」を問うと、注力すべき場所が見えてくる。

1.1.3 ソフトウェア工学とは何か

ソフトウェア工学の定義として広く参照されるのは、米国電気電子学会 (Institute of Electrical and Electronics Engineers, IEEE) の用語標準である。そこでは、ソフトウェアの開発・運用・保守に対する、体系的・規則的・定量的なアプローチの適用、と定義される [IEE90]。鍵となるのは三つの形容詞である。**体系的**とは、場当たりでなく筋道立てて進めること。**規則的**とは、個人の勘ではなく共有された規律に従うこと。**定量的**とは、品質や進捗を測り、数値で語ることである。

ソフトウェア工学が扱う知識の全体像は、ソフトウェア工学知識体系 (Software Engineering Body of Knowledge, SWEBOK) として整理されている [WO24]。図 1.2 にその知識領域を示す。要求から保守までの開発工程の中核に加え、アーキテクチャ、セキュリティ、運用、構成管理、品

要求 (Requirements)	アーキテクチャ (Architecture)	設計 (Design)	構築 (Construction)	テスト (Testing)	保守 (Maintenance)
構成管理 (Config. Mgmt.)	工学的管理 (Mgmt.)	プロセス (Process)	モデルと手法 (Models)	品質 (Quality)	セキュリティ (Security)
運用 (Operations)	専門職実践 (Practice)	経済 (Economics)	計算機基礎 (Computing)	数学基礎 (Math)	工学基礎 (Engineering)

図 1.2 SWEBOK V4 の 18 知識領域。上段の橙色は開発工程の中核，中段と下段は管理・品質・運用・基礎系である。本書は全領域を一律には扱わず，比重に応じて取捨する。

質，プロセス，さらには経済や数学といった基礎までが含まれる。本書はこのすべてを同じ重さで扱うわけではないが，現代の事例で語り直しながら，重要性を増す領域を選んで深める。

数値で見る姿勢は，ソフトウェア工学の強みである。大規模プロジェクトでは，潜在欠陥数や欠陥除去率を推定し，レビューやテストでどれだけ欠陥を取り除けたかを追うことがある。SAFEGUARD 弾道ミサイル防衛システムのような初期の巨大防衛システム開発は，工程管理と品質測定を強く意識させた歴史的な事例である。ただし，数値は判断を助ける道具であり，開発を自動的に成功させる保証ではない。要求の曖昧さ，関係者間の誤解，経験差，組織文化は，欠陥数だけでは見えない。定量化は必要である。同時に，定量化できない部分を見落とさないこともソフトウェア工学である。

ソフトウェア工学は **工学** (engineering) である。工学は，無制限の資源で理想を追うのではなく，限られた時間，費用，人員のもとで，十分に良い解を選ぶ意思決定である。完璧なソフトウェアを目指すのではなく，与えられた制約のなかで品質とコストの釣り合いを取る。どの候補を採用し，どれを捨てるかを根拠で判断する。この態度が，本書全体の土台である。

この基礎概念を AI 駆動開発でどう使うかは第 II 部 (特に第 13 章) で扱う。

演習

演習 1-1 (ソフトウェア危機との対応)。過去 5 年以内に報道された大規模なシステム障害事例の一つを選び，1968 年のソフトウェア危機と何が同じで何が違うかを論じよ。とくに，本質的複雑性と偶発的複雑性のどちらに起因する障害かを考察せよ。

第2章 品質，信頼性，安全性

本章の到達目標.

知識 品質モデル，信頼性，可用性，安全性，保守性，専門職倫理の基本語を説明できる.

適用 既存サービスを品質特性で評価し，障害時の振る舞いをフォールトトレラント，フェールソフト，フェールセーフ，フルプルーフの観点で分類できる.

実践 小規模プロジェクトに対して，品質目標と障害時方針を明文化できる.

2.1 本章の位置づけ

第1章では，ソフトウェア工学を，社会で動き続けるソフトウェアを作るための工学として位置づけた．本章では，その「良く作る」を測る言葉を導入する．ソフトウェアは，単に動けば十分とは言えない．正しく動くこと，必要なときに使えること，壊れても被害を限定できること，変更に耐えること，利用者の権利を損なわないことが求められる．これらをまとめて扱う概念が **品質** である．

品質を感想で終わらせず，観点，基準，測定方法，合意された優先順位を持つ工学的対象として扱う．同じソフトウェアでも，ゲーム，医療機器，銀行勘定系，研究室の備品予約では，重視すべき品質が異なる．したがって品質を議論するときは，何をよいとみなすかを先に分解しておく必要がある．

2.2 品質モデル

品質モデル は，「良いソフトウェア」を分解して語るための道具である．古典的には Boehm らの品質モデルが知られ，現在は ISO/IEC 25010 が広く参照される [Boe78; 11; 23]. ISO/IEC 25010:2011 では，機能適合性，性能効率性，互換性，使用性，信頼性，セキュリティ，保守性，移植性の 8 特性が示された．2023 年改訂 (ISO/IEC 25010:2023) では，安全性 (safety) が独立した特性として加わり，使用性は対話能力 (interaction capability) へ，移植性は柔軟性 (flexibility) へ再編され，全体で 9 特性となった．標準は更新されるため，本書では名称の暗記よりも，観点を分けて議論する姿勢を重視する．次の表 2.1 は，2023 年版の区分に沿って観点の読み方を示す．

品質特性は互いに影響する．性能を上げるためにキャッシュを増やすと，一貫性やセキュリティが難しくなることがある．可用性を上げるために冗長化すると，構成が複雑になり保守性が下がることもある．したがって品質設計では，まず優先順位を定める．「全部を最高にする」は要求では

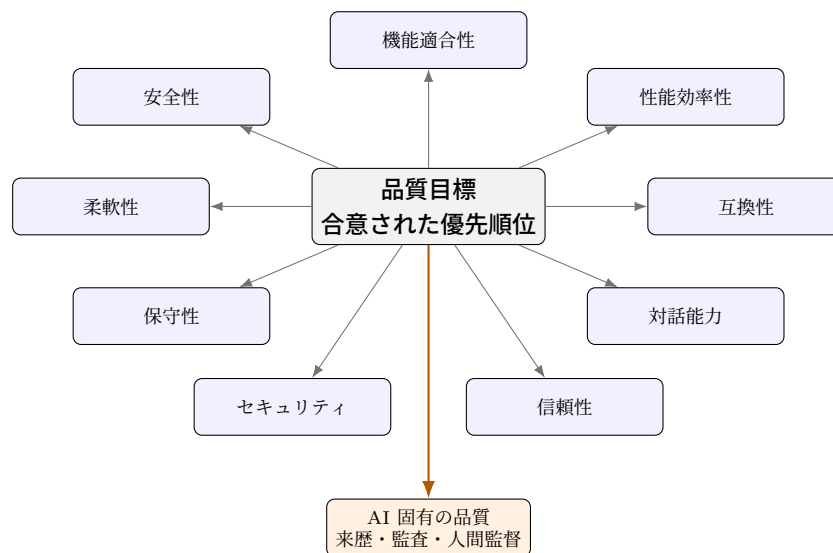


図 2.1 品質を複数の観点のバランスで評価する

表 2.1 ISO/IEC 25010:2023 の品質特性の読み方

観点	問い	研究室備品予約システムの例
機能適合性	必要な機能を満たすか	予約、取消、通知が要求どおり動く
性能効率性	時間と資源を浪費しないか	混雑時も応答が遅すぎない
互換性	他システムと共存できるか	大学認証と連携できる
対話能力	利用者が迷わずやり取りできるか	初回利用者でも予約手順を理解できる
信頼性	必要なときに動くか	障害時もデータを失わない
セキュリティ	不正利用を防げるか	権限のない予約変更を拒む
保守性	変更しやすいか	新しい備品種別を安全に追加できる
柔軟性	環境や要求の変化に適応できるか	ローカルからクラウドへ移し、利用規模に合わせて伸縮できる
安全性	人や財産や環境への許容できない害を防げるか	二重予約や危険操作を安全側へ止める

なく願望である。

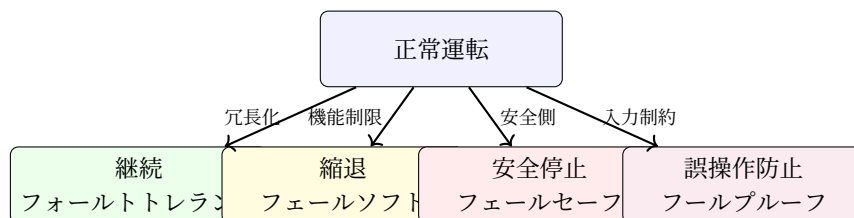
2.3 信頼性，可用性，安全性

信頼性 は、定められた条件で定められた期間、故障せずに機能する性質である。**可用性** は、必要なときに利用可能である性質である。**安全性** は、故障や誤操作があっても、人や財産や社会へ許容できない被害を与えない性質である。これらは似ているが同じではない。短時間の停止を許せるサービスでも、誤った医療線量を出す機器は許されない。また、停止しないことより、安全に停止することが重要な場面もある。

障害に備える設計思想には、古くから使われてきた基本語がある。これらは古い用語に見えるが、現代のクラウド設計にも対応する。

表 2.2 古典的な信頼性設計語と現代の対応

用語	意味	現代的な例
フォールトトレラント	故障があっても機能を継続する	冗長化, レプリカ, 複数 アベイラビリティゾーン (Availability Zone, AZ) 配置
フェールソフト	一部機能を落として全体停止を避ける	推薦を止めても検索を残す
フェールセーフ	故障時に安全側へ移る	決済確認不能なら課金しない
フルブルーフ	誤操作をしにくくする	危険操作に確認と権限を置く
デュアルシステム	二系統が同じ処理を担う	二重化した制御系で照合する
ホットスタンバイ	待機系を即時切替可能にする	稼働中の予備サーバへ自動切替する



障害時の振る舞いを先に決めると、実装と運用の判断が揃う

図 2.2 障害時方針は継続，縮退，安全停止，誤操作防止に分けて考える

現代の分散システムでは、これらの考え方は **レジリエンス**，**サーキットブレーカ**，**タイムアウト**，**リトライとバックオフ**，**フォールバック**，**グレースフルデグラデーション** といった語で現れる。用語は変わっても、障害を前提にして被害を限定するという骨格は変わらない。

2.4 測定とその限界

品質を議論するには測定が必要である。例えば、平均故障間隔、平均修復時間、テストカバレッジ、循環的複雑度、欠陥密度、レビュー指摘数、ユーザ離脱率などが使われる。McCabe の循環的複雑度は、制御フローの分岐に基づいてテストや保守の難しさを見積もる代表的な指標である [McC76]。

ただし、測定値は品質そのものではない。カバレッジが高くても、重要な境界条件を試していなければ欠陥は残る。レビュー指摘数を評価に使うと、軽微な指摘を増やす行動が誘発されることもある。指標が目標になると、指標は良い測定でなくなる。本書では、数値を捨てるのではなく、数値が何を見落としているかを常に問う。

2.5 事故から学ぶ

安全性の教育でよく参照される事例に Therac-25 の放射線治療事故がある。Leveson らの調査は、単一のバグではなく、ソフトウェア、ユーザインタフェース、組織、検証体制が絡み合って事故が起きたことを示した [LT93]。安全性は、コードの正しさだけでなく、システム全体の制御構造として考える必要がある [Lev11]。

この見方は、小規模な業務システムにも関係する。予約システムで二重予約が起きたとき、原因は重複判定関数だけとは限らない。要求が曖昧だった、画面が誤操作を誘った、データベース (Database, DB) 制約がなかった、レビューで境界条件を見なかった、運用で取消手順が決まっていなかった、という複数の要因が重なることがある。ソフトウェア工学は、この連鎖を切るために品質を先に言葉にする。

2.6 品質目標と障害時方針

品質目標は、設計とテストの前に、何を守るかを明文化するための基準である。例えば 研究室 備品予約システムなら、次のように書ける。

可用性 平日 9 時から 18 時は予約登録を受け付ける。

安全性 予約の二重登録を防ぎ、競合時は片方を明示的に拒否する。

セキュリティ 備品管理者だけが備品を追加・停止できる。

保守性 備品種別の追加時に既存予約の扱いを変更しなくてよい構造にする。

品質設計では、正常系だけでなく障害時の物語を書く。ネットワークが切れたらどうするか。外部認証が落ちたらどうするか。管理者が誤操作したらどう戻すか。このような問いに答えることで、フェールソフト、フェールセーフ、フォールバックの配置が見える。

表 2.3 研究室備品予約システムにおける障害時方針の例

事象	方針	品質観点
認証基盤が停止	新規予約を停止し、既存予約閲覧のみ 許す	フェールソフト，可用性
二重予約が発生しそう	片方を確定し、もう片方に再選択を 促す	一貫性，安全性
通知サービスが失敗	予約登録は保留せず、画面と履歴で確 認できるようにする	フェールソフト，可用性
管理者が誤って備品削除	論理削除にして復元可能にする	フルプルーフ，保守性

2.7 専門職倫理

専門職の倫理綱領は、「良い人であれ」という精神論を超えて、利害が衝突したときに何を優先すべきかを考えるための実務的な基準になる。利用者の安全、プライバシー、説明責任、公平性、専門能力の限界、公共の利益は、ソフトウェア技術者が避けて通れない。AI を含む責任とガバナンスは第 14 章で扱うが、その前提は本章の品質と安全性である。

演習

演習 2-1 (品質特性の分類). 日常的に使うアプリを一つ選び、品質特性を 5 個以上用いて良い点と問題点を分類せよ。主観的評価だけでなく、観測できる事実を一つ以上含めること。

演習 2-2 (信頼性設計語の対応). フォールトトレラント、フェールソフト、フェールセーフ、フルプルーフの例を、自分が知っているシステムから一つずつ挙げよ。その例が本当に該当する理由も書け。

演習 2-3 (品質目標). 研究室備品予約システムに「予約理由を要約して表示する機能」を追加する場合、品質目標、禁止すべき振る舞い、障害時方針を記述せよ。

第3章 要求工学

本章の到達目標.

知識 ユーザ要求, システム要求, 機能要求, 非機能要求, 要求構文の簡易手法 (Easy Approach to Requirements Syntax, EARS), ユースケース, 受入基準を説明できる.

適用 曖昧な要求文を検証可能な要求へ書き直し, 利害関係者へ確認すべき質問を作れる.

実践 小規模プロジェクトについて, 要求一覧, ユースケース, 受入基準を含む要求仕様の初稿を作れる.

3.1 本章の位置づけ

要求定義は, 作るべきものを決める活動である. 実装技術が高くても, 要求を誤れば正しい失敗を高速に作るだけである. 要求は, 利害関係者との合意であり, 合意には責任が伴う. したがって要求工学では, 発話や要望をそのまま書き写すのではなく, 何が決まっていて, 何が未決で, どの条件を満たせば受け入れられるかを明確にする.

本章では, 古典的な要求工学を軽量に扱う. データフロー図 (Data Flow Diagram, DFD) などの古い分析表記は歴史的意義に留め, 主軸は自然言語要求, ユースケース, 受入基準, 後続の設計へ接続できるテキスト表現とする. AI による要求分析補助は第 13 章で扱う. ここでは, AI を使わなくても必要な基礎を固める.



図 3.1 要求は発話から仕様へ段階的に変換される

3.2 要求の種類

ユーザ要求 は, 利用者や顧客の言葉で述べられる要求である. **システム要求** は, 開発者が設計と実装へ接続できる形に整理した要求である. また, **機能要求** はシステムが何をするかを述べ, **非機能要求** は性能, 信頼性, 使用性, セキュリティ, 法令遵守など, どのように満たすかを述べる. ISO/IEC/IEEE 29148:2018 は要求工学のプロセスと要求記述の考え方を整理している [18].

3.3 良い要求文

良い要求文は、曖昧でなく、検証可能で、一貫し、追跡できる。「高速に」「使いやすく」「安全に」は、そのままでは要求にならない。何を、誰が、いつ、どの条件で、どの程度満たすかを書く必要がある。

要求文を構造化する方法として EARS がある [Mav+09]。EARS は、イベント、状態、任意条件、望ましくない条件などに応じて要求文の型を使い分ける。厳密な形式仕様ほど重くなく、自然言語だけより曖昧さを減らせるため、小規模な要求整理にも使いやすい。

3.4 ユースケースと受入基準

ユースケース は、アクタが目的を達成するための相互作用を記述する方法である [Coc00]。ユースケースは、画面の説明ではなく、利用者の目的とシステムの応答を書く。受入基準は、要求が満たされたと判断する条件である。本書では、要求からテストへ接続しやすいよう、受入基準を Given, When, Then の形で書く。

```
1 Given a microscope is available from 10:00 to 12:00
2 When a student reserves it from 10:30 to 11:30
3 Then the reservation is accepted
4
5 Given a microscope is already reserved from 10:00 to 12:00
6 When another student reserves it from 11:00 to 12:30
7 Then the reservation is rejected with a conflict message
```

Listing 3.1 受入基準の例

受入基準は、後のテストケースの種になる。この時点で境界条件や例外を考えることで、設計と実装の手戻りを減らせる。

表 3.1 要求の分類例

種類	例	注意点
ユーザ要求	利用者が備品を簡単に予約したい	「簡単」の意味を確認する
システム要求	利用者は空き時間帯を選択して予約を登録できる	入力、出力、例外を定める
機能要求	予約登録、取消、承認、通知を備える	画面ではなく振る舞いを書く
非機能要求	予約登録は 3 秒以内に完了する	測定条件を添える
制約	大学の認証基盤を利用する	外部依存を明示する

3.5 研究室備品予約システムの最小導入

研究室備品予約システムは、組織内の備品を利用者と管理者が予約・管理する小規模システムである。共通題材として、後続章で設計、実装、データベース、テストへ展開する。

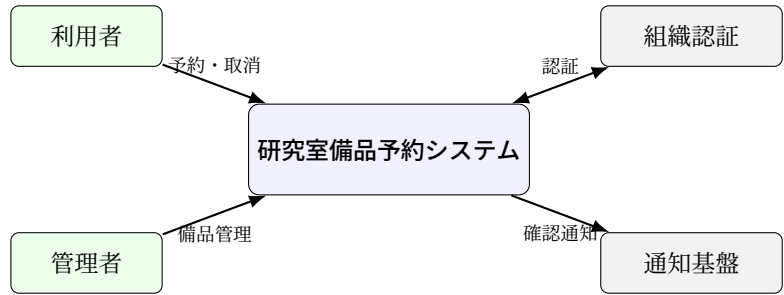


図 3.2 研究室備品予約システムの文脈図

初期要求メモを次のように置く。

組織の利用者が、顕微鏡や高性能 GPU 端末などの備品を簡単に予約できるようにしたい。予約が重ならないようにし、管理者は備品の追加や停止ができるようにする。必要なら通知も出したい。

このメモには曖昧が多い。「利用者」とは組織のメンバーだけか、外部協力者も含むか。「簡単」とは何ステップか。「予約が重ならない」とは同じ備品だけか、同じ利用者の重複も含むか。「必要なら通知」とは誰に、いつ、どの媒体で通知するのか。要求工学では、このような未決事項を隠さず、確認質問として残す。

3.6 要求一覧の初稿

この段階では完全性を求めない。むしろ、未決事項が見える形で残す。要求仕様は一度で完成する文書ではなく、合意の履歴である。

表 3.2 EARS の簡略パターン

型	形	例
通常	システムは X する	システムは予約一覧を表示する
イベント	Y のとき、システムは X する	利用者が予約を送信したとき、システムは競合を検査する
状態	Z の間、システムは X する	貸出停止中の間、システムは新規予約を拒否する
任意条件	W が有効なら、システムは X する	通知設定が有効なら、システムは確認メールを送る
例外	E が発生したとき、システムは X する	認証に失敗したとき、システムは予約登録を拒否する

3.7 要求から設計へ

要求は第 4 章の設計へ接続される。FR-03 は、予約の時間範囲と競合判定を設計対象にする。FR-04 は、利用者の役割と権限モデルを必要にする。NFR-02 は、データベース設計で監査ログを扱う理由になる。このように、要求は後続成果物の根拠である。設計案を作る場合も、根拠となる要求 ID を対応させる。

演習

演習 3-1 (曖昧語抽出). 与えられた要求メモから、曖昧語を 10 個以上抽出し、利害関係者に聞く確認質問を書け。

演習 3-2 (EARS 変換). 「ユーザは備品を簡単に予約できる」を EARS の形で 3 文以上に分解せよ。通常、イベント、例外の型を少なくとも一つずつ含めること。

演習 3-3 (受入基準). 研究室備品予約システムの FR-03 に対して、成功例、失敗例、境界例を Given, When, Then の形で書け。

表 3.3 研究室備品予約システムの要求初稿

ID	要求	受入基準の方向
FR-01	利用者は備品の空き時間を閲覧できる	日付と備品で空き枠を確認できる
FR-02	利用者は空き時間に予約を登録できる	競合がなければ予約が保存される
FR-03	システムは同一備品の重複予約を拒否する	時間範囲が重なる予約を拒む
FR-04	管理者は備品を追加、停止、編集できる	管理者以外は操作できない
NFR-01	予約登録は通常 3 秒以内に完了する	測定条件を定めて確認する
NFR-02	予約変更の履歴を追跡できる	変更者と時刻を残す

第4章 モデリングと設計

本章の到達目標.

知識 オブジェクト指向, UML 主要図, SOLID, 代表的デザインパターン, ペトリネットの要素と発火規則を説明できる.

適用 小規模システムをクラス図, シーケンス図, 状態機械, ペトリネットで表現し, 設計上の責務と並行性を説明できる.

適用 設計案を SOLID とパターンの観点で分析し, 責務分割と資源競合の問題点を指摘できる.

4.1 本章の位置づけ

要求は, 作るべきものを述べる. 設計は, それをどのような構造と振る舞いで実現するかを決める. 要求からいきなりコードへ進むと, 責務, 状態, データ, 例外, 並行性がコード中に散らばる. 設計は, これらを実装前に見える形へ置く活動である.

本章では, 現代の主流であるオブジェクト指向と UML を中心に扱う. DFD や構造化分析は歴史的背景として軽く触れるに留める. ただし, ペトリネットは残す. 日常の Web 開発で毎日描く図とは限らないが, 並行システム, 資源競合, デッドロックを理解する基礎として有用である.

設計モデルを学ぶ目的は, 図をきれいに描くことよりも, 責務の置き場所と守るべき制約を言葉にできるようになることである. UML で責務, 多重度, 状態遷移を示し, ペトリネットで資源競合を示せば, システムの構造と並行性を実装前に検討できる.

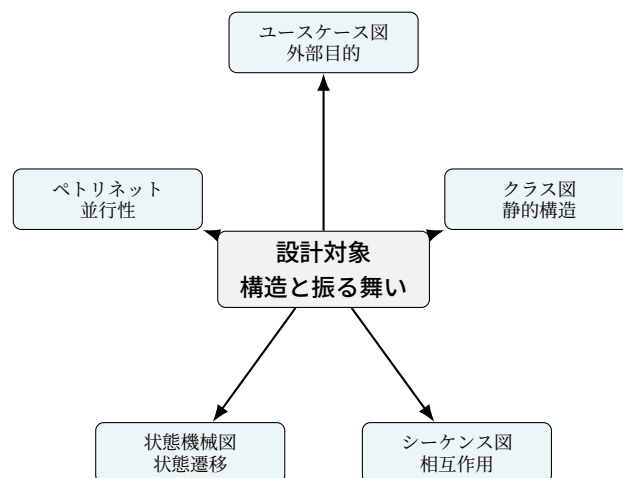


図 4.1 設計では複数の見方を使い分ける

4.2 モデルは目的に応じて現実を削る

モデル は、対象の一部を目的に応じて切り出す表現である。目的に応じて重要な部分を選び、それ以外を捨てる。クラス図は静的構造を示すが、時間順の相互作用は示しにくい。シーケンス図は相互作用を示すが、状態の全体像は示しにくい。状態機械は状態遷移を示すが、データ構造は示しにくい。したがって、一つの図ですべてを説明しようとししない。

古い教科書では、構造化分析や DFD が大きく扱われることがある。DFD はデータの流れを理解する道具として今でも読めるが、本書では主軸にしない。現代のソフトウェア教育では、オブジェクト、責務、インタフェース、状態、データモデルを組み合わせる方が実装へ接続しやすいからである。第 3 章では自然言語要求、ユースケース、受入基準を主軸にした。ここでは歴史的な表記法を、設計モデルを選ぶための背景として最小限に整理する。

表 4.1 歴史的・基礎的なモデル表記の使い分け

表記	主に表すもの	本書での位置づけ
DFD コントロールフロー	処理、データ、データの流れ 条件分岐と制御の流れ	歴史的背景。主表記にはしない アクティビティ図やコード読解へ 接続
有限状態機械	状態と遷移	UML 状態機械図の基礎
ER モデル	実体と関連	第 6 章のデータ設計へ接続
オブジェクト指向モデル	オブジェクト、責務、関連	本章の主軸
ペトリネット	並行性、資源競合、発火	本章と付録 A で扱う

4.3 オブジェクト指向の基本

オブジェクト指向は、データと振る舞いを持つオブジェクトの協調としてシステムを考える方法である。代表的な概念は、カプセル化、抽象化、継承、ポリモーフィズムである。

カプセル化 内部状態を直接触れせず、公開された操作を通じて扱う。

抽象化 重要な性質だけを取り出し、詳細を隠す。

継承 既存の型の性質を引き継いで特殊化する。ただし濫用すると結合が強くなる。

ポリモーフィズム 同じインタフェースで異なる実装を扱う。

設計教育では、継承の使用よりも、責務を適切に分けることが重要である。研究室備品予約システムなら、予約、備品、利用者、通知、権限、監査ログを同じクラスへ詰め込むと変更に弱くなる。

4.4 UML 主要図

統一モデリング言語 (Unified Modeling Language, UML) は、ソフトウェア設計を図で表すための標準的な記法である [17]。本書では全図種を網羅しない。本文で扱うのは、クラス図、シーケンス図、ユースケース図、状態機械図、アクティビティ図の 5 種である。その他の図は付録 C で扱う。

表 4.2 本文で扱う UML 主要図

図	表すもの	研究室備品予約システムの例
ユースケース図	外部アクタと目的	利用者が備品を予約する
クラス図	型、属性、関連、多重度	Reservation と Equipment の関連
シーケンス図	時間順のメッセージ	予約登録時の競合確認
状態機械図	状態と遷移	予約が仮予約、確定、取消へ移る
アクティビティ図	処理の流れと分岐	管理者承認の業務手順

4.4.1 多重度を読む

UML クラス図では、関連の端に多重度を書く。多重度は実装の配列や外部キーへ直結するため、読み間違えると設計が壊れる。

表 4.3 UML 多重度の表記と読み方

表記	意味	例
1	ちょうど一つ	予約は一つの備品に対応する
0..1	なし、または一つ	予約は承認者を持たない場合がある
0..*	なし、または複数	備品は複数の予約を持ち得る
1..*	一つ以上	利用者は少なくとも一つの所属を持つ
*	0..* と同じ意味で使われる	備品に予約が複数付く

4.5 設計原則とパターン

設計原則は、コードを書く前に変更に強い構造を考えるための言葉である。SOLID は代表的な原則群であり、単一責任、開放閉鎖、リスコフ置換、インタフェース分離、依存関係逆転を含む [Mar02]。四人組 (Gang of Four, GoF) のデザインパターンは、生成、構造、振る舞いに関する再利用可能な設計語彙を与える [Gam+94]。

ただし、パターンを使うだけで良い設計になるとは限らない。Factory, Strategy, Observer などの名前を暗記するより、どの変更に備えているのかを説明できることが重要である。デザインパターンは抽象化を増やす合図というより、変更点、責務、依存関係を説明するための共通語彙である。小さなシステムでは、単純さも品質である。

4.6 並行性モデル: ペトリネット入門

4.6.1 なぜペトリネットを学ぶか

ペトリネットは、並行性、資源競合、同期、デッドロックを表すための数理モデルである [Pet62; Mur89]. Web アプリケーションを作るとき、毎回ペトリネットを描く必要はない. しかし、予約重複、排他制御、ジョブキュー、ワークフロー、分散処理を理解するとき、「状態がどこにあり、どの事象で移るか」を考える力が役立つ.

4.6.2 四つの要素

ペトリネットの基本要素は、プレース、トランジション、アーク、トークンである. プレースは状態や資源の置き場所、トランジションは事象や処理、アークは接続、トークンは現在の資源数や状態を表す. 図 4.2 に、それぞれの見た目を示す. プレースは丸、トランジションは棒、アークは矢印、トークンはプレースの中の黒丸で描く.

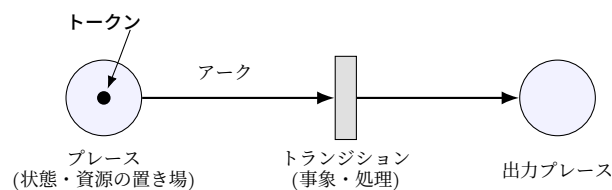


図 4.2 ペトリネットの四つの基本要素. 丸で表すプレースに置かれた黒丸のトークンが、矢印のアークでつながれた棒状のトランジションの発火で移動する.

ある時点で各プレースにあるトークンの分布を **マーキング** と呼ぶ. マーキングはシステムの「いまの状態」であり, M_0 を初期マーキングと書く. 各プレースのトークン数を並べたベクトルで表すことが多い. 例えばプレースが二つなら, $M_0 = (1, 0)$ は最初のプレースにトークンが一つ, 二つ目は空であることを表す.

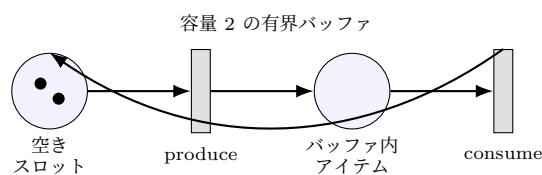


図 4.3 有界バッファを表す最小ペトリネット

4.6.3 発火規則

トランジションは、入力側のすべてのプレースに必要なトークンがあるとき **発火可能** になる。発火すると、入力プレースからトークンを取り、出力プレースへトークンを置く。この単純な規則により、同期や競合を表せる。

発火はマーキングを変える。図 4.3 の有界バッファで、プレースを空きスロット、バッファ内アイテムの順に並べると、初期マーキングは $M_0 = (2, 0)$ である。ここで produce が一度発火すると、空きスロットからトークンが一つ減り、バッファ内アイテムへ一つ増えて $M_1 = (1, 1)$ になる。さらに consume が発火すると $M_2 = (2, 0)$ に戻る。発火できるトランジションが複数あるとき、どれが発火するかは規則では決めない。この非決定性こそが、並行システムで「起こりうる順序」を網羅的に考えるための表現力である。

図 4.3 では、空きスロットがあるときだけ produce が発火し、バッファ内アイテムがあるときだけ consume が発火する。空きスロットもアイテムもトークンとして表すため、バッファ容量の制約が自然に表れる。

4.6.4 排他制御の例: 備品の取り合い

ペトリネットは、研究室備品予約システムのような資源の取り合いを素直に表せる。図 4.4 は、一つの備品を利用者 A と利用者 B が同時に確保しようとする状況である。プレース「備品利用可」に置かれた一つのトークンが、その備品が今は空いていることを表す。

このトークンが一つしかない点が鍵である。「A が確保」と「B が確保」は、どちらも「備品利用可」を入力プレースに持つ。したがって初期マーキングではどちらも発火可能だが、一方が発火するとトークンが消費され、もう一方は発火不能になる。図の右側では、A が確保を発火した結果、トークンが「A が利用中」へ移り、「B が確保」は入力トークンを失って発火できない。これがペトリネットによる **相互排他** の表現である。

このモデルは、第 3 章の要求 FR-03 (同一備品の重複予約を拒否する) が、なぜ単一の資源トークンとして設計できるかを示す。実装では、この「トークンが一つ」という不変条件を、データベースの一意制約やロックで保証する (第 6 章)。設計の段階でペトリネットとして描いておくと、競合が起こりうる箇所と、それを守るべき不変条件が明確になる。

4.6.5 デッドロックの直観

どのトランジションも発火できないマーキングに到達すると、システムは停止する。これが **デッドロック** の直観である。実システムでは、二つの処理が互いに相手の資源解放を待つ、ジョブキューが満杯で新規ジョブを受け取れない、承認フローが誰にも割り当てられていない、といった形で現れる。ペトリネットは、このような停止状態を図として見つける助けになる。

4.6.6 現代での位置づけ

ペトリネットは、日常の UI 設計や Web API 設計の主表記ではないが、並行性を持つシステムを形式的に考える入口として価値がある。状態機械図や ビジネスプロセスモデル表記 (Business Process Model and Notation, BPMN)、ワークフローエンジン、分散システムの設計にも思想が伝わる。本書では本文で直観を学び、詳細な到達可能性解析や不変量は付録 A へ送る。

演習

演習 4-1 (責務分割). Reservation クラスに通知、権限、競合判定、保存処理をすべて入れた設計案の問題点を、SOLID の観点で説明せよ。

演習 4-2 (ペトリネット). 図 4.3 の初期マーキングから、produce, consume を 3 回以上発火させ、マーキングの変化を書け。また、デッドロックが起きる条件を説明せよ。

ここで学んだ設計の基礎概念を AI 駆動開発でどう使うかは第 II 部 (特に第 13 章) で扱う。

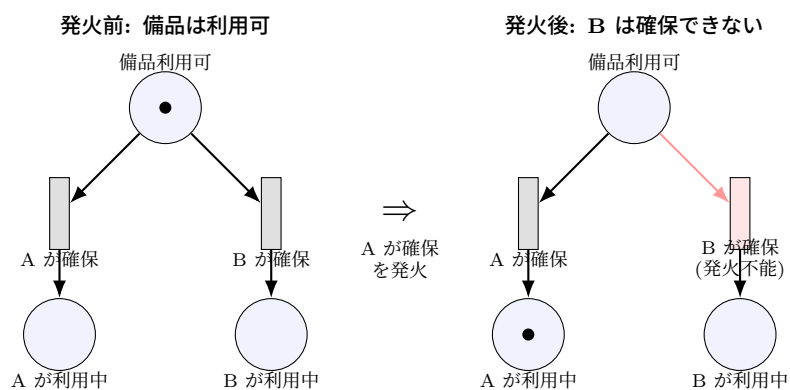


図 4.4 備品予約の排他制御。「備品利用可」のトークンが一つだけのため、A が確保を発火するとトークンが移り、B は確保できなくなる。これが相互排他最小モデルである。

第 5 章 Python プログラム読解

本章の到達目標.

知識 Python の名前とオブジェクト, 参照と束縛, 共有渡し, 浅いコピー, 深いコピー, 型ヒント, dataclass, 例外, 非同期の入口を説明できる.

適用 リスト操作, 関数引数, `copy.deepcopy` を含む Python コードの実行結果を予測できる.

実践 型ヒント, dataclass, 例外処理を備えた小規模 Python コードを, 意図が読み取れる形に設計できる.

5.1 本章の位置づけ

本書はプログラミング入門を本文で繰り返さない. 変数, 条件分岐, 繰り返し, 関数定義の初歩は付録 E の事前課題に置く. 本章で扱うのは, ソフトウェア工学の実践でつまづきやすい Python の意味論である. 他言語のメモリ管理, 参照, 所有権との比較は付録 B にまとめた.

特に Python では, 名前とオブジェクトの関係, 可変オブジェクトの共有, 浅いコピーと深いコピーの違いを誤解すると, 見た目は短いが危険なコードになる. 本章では, 実行できるコードより先に, 読めるコードと説明できるコードを重視する. 意味論を正しく押さえることが, 後で他者のコードを読む土台になる.

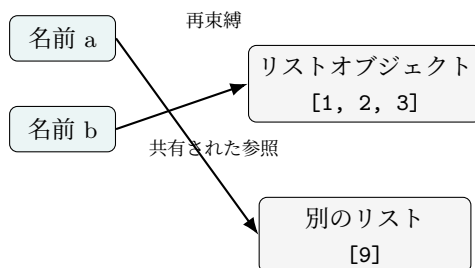


図 5.1 Python の名前は箱よりもオブジェクトへの参照として考える

5.2 名前とオブジェクト

Python の代入では, 箱に値を入れるという比喩が誤解を招く. 名前をオブジェクトへ束縛する操作である. 同じオブジェクトを複数の名前が参照することがある. 整数や文字列のような不変オ

ブジェクトでは問題が見えにくいですが、リストや辞書のような可変オブジェクトでは共有が結果に現れる。

```
1 a = [1, 2]
2 b = a
3 b.append(3)
4 a = [9]
5 print(b)
```

Listing 5.1 名前の再束縛とオブジェクトの共有

この出力は [1, 2, 3] である。b = a により a が指していたリストを b も指す。その後 b.append(3) は同じリストを変更する。最後の a = [9] は a を別のリストへ再束縛するだけで、b が指すリストを変えない。

5.3 可変性と共有渡し

Python の関数呼び出しは、よく **共有渡し** と説明される。引数として渡されたオブジェクトを関数内の名前が参照する。関数内でそのオブジェクトを変更すれば呼出側にも見える。一方、関数内の名前を別オブジェクトへ再束縛しても、呼出側の名前は変わらない。

```
1 def add_item(xs):
2     xs.append("A")
3
4 def replace_list(xs):
5     xs = ["B"]
6
7 items = []
8 add_item(items)
9 replace_list(items)
10 print(items)
```

Listing 5.2 共有渡しの実行結果

出力は ['A'] である。add_item は呼出側と共有されたリストを変更する。replace_list は関数内の名前 xs を新しいリストへ向けるだけである。この区別は、小規模プロジェクトの欠陥解析でも重要である。

5.4 浅いコピーと深いコピー

浅いコピー は外側のコンテナを複製するが、中にあるオブジェクトは共有する。**深いコピー** は入れ子になったオブジェクトも再帰的に複製する。

```
1 import copy
2
3 original = [[1, 2], [3, 4]]
```

```

4 shallow = list(original)
5 deep = copy.deepcopy(original)
6
7 shallow[0].append(9)
8 deep[1].append(8)
9
10 print(original)
11 print(shallow)
12 print(deep)

```

Listing 5.3 浅いコピーと深いコピー

`original` の一つ目の内側リストには 9 が追加される。 `shallow[0]` と `original[0]` が同じリストを指しているからである。一方、 `deep[1]` の変更は `original` に影響しない。深いコピーは便利だが、巨大なデータ構造では重い。コピーする前に、そもそも共有してよい設計かを考える必要がある。

5.5 型ヒントと実行時検査を分ける

型ヒントは、プログラムの意図を人間とツールへ伝える注釈である [RLL14; Sla19]。標準の Python 実行系は、型ヒントだけで実行時に型を拒否しない。したがって、型ヒントは契約の記述であり、必要に応じて静的解析、テスト、実行時検査と組み合わせる。

```

1 from dataclasses import dataclass
2 from datetime import datetime
3
4 @dataclass(frozen=True)
5 class TimeInterval:
6     start: datetime
7     end: datetime
8
9 def overlaps(a: TimeInterval, b: TimeInterval) -> bool:
10     return a.start < b.end and b.start < a.end

```

Listing 5.4 型ヒントと `dataclass` を使った範囲表現

`dataclass(frozen=True)` は、値オブジェクトとしての意図を明示する。時間範囲を後から書き換ええない設計にすれば、重複判定の推論が容易になる。Python では可変性を完全には封じられない場合もあるが、設計意図をコードへ寄せることには価値がある。

5.6 例外と失敗の表現

例外は、通常の戻り値では表しにくい失敗を伝える仕組みである。ただし、すべてを例外にすればよいとは限らない。入力検証の失敗、権限不足、外部 API のタイムアウト、データベースの一意制約違反は、呼出側がどう扱うべきかが異なる。

5.7 非同期処理の入口

Python の `async` と `await` は、待ち時間の多い処理を効率よく扱うための仕組みである。データベース、ファイル I/O、ネットワーク呼び出しは待ち時間を含むことが多い。非同期処理は性能改善の道具だが、同時にタイムアウト、キャンセル、順序、例外処理を難しくする。最初から複雑な処理を非同期で組まず、まず同期版で正しさを確認し、必要な箇所だけ非同期化するのがよい。

ここまで扱った Python の意味論は、どのような開発でも共通する基礎である。これらの基礎概念を AI 駆動開発でどう使うかは第 II 部 (特に第 13 章) で扱う。

演習

演習 5-1 (実行結果予測). リストを関数へ渡し、関数内で `append` と再束縛をするコードを自作し、出力を予測してから実行せよ。予測が外れた場合は、どの名前がどのオブジェクトを指していたかを図示せよ。

演習 5-2 (深いコピー). 入れ子のリストと辞書を含むデータ構造を作り、浅いコピーと `copy.deepcopy` の結果の違いを説明せよ。

演習 5-3 (型ヒントと `dataclass`). `TimeInterval` に状態を表す `Enum` を追加し、確定、取消、拒否の状態遷移を関数として実装せよ。型ヒントを付け、不正な状態遷移を例外または検証結果で表し、状態が不正に書き換わらない設計を説明せよ。

表 5.1 失敗の種類と表現

失敗	典型的な表現	注意点
入力不正	<code>ValueError</code> または検証結果	利用者へ戻すメッセージを分ける
権限不足	独自例外またはエラーコード	監査ログを残す
外部 API 失敗	タイムアウト例外、リトライ対象	無限リトライを避ける
競合	データベース制約違反、ドメイン例外	直前の確認だけに頼らない

第 6 章 データモデリングとデータベース

本章の到達目標.

知識 関係モデル, 関係代数, SQL, ER モデル, 主キー, 外部キー, 関数従属性, 第 1 から第 3 正規形, ACID を説明できる.

適用 業務記述から ER 図と関係スキーマを作り, GROUP BY, HAVING, AVG を含む SQL を書く. 表を第 3 正規形まで分解できる.

実践 データ形状, 更新頻度, 一貫性要求に基づいて適切なデータ管理方式を選べる.

6.1 本章の位置づけ

第 4 章では, クラス図や状態機械を使って構造と振る舞いを表した. 本章では, 永続化されるデータを扱う. データベースは単なる保存場所ではなく, 制約, 一貫性, 検索, 更新, 権限, 履歴を担うソフトウェア基盤である.

関係データベースの基礎は, 特定の世代の技術に依存しない普遍的な土台である. 人間が関係モデル, 正規化, 集計, トランザクションを読めることは, データを扱うあらゆる場面で前提となる. 本章では, RAG やベクトル検索を扱わない. それらは第 11 章で知識接続の技術として扱う. ここでは, 業務データの一貫性を守るための 関係データベース (Relational Database, RDB) を中心に学ぶ.

6.2 データ独立性

データベースの重要な役割は, プログラムとデータの物理的保存方法を分離することである. これを **データ独立性** と呼ぶ. プログラムがファイル形式や配置を直接知っていると, 保存方式を変えるだけで多くのコードが壊れる. データベース管理システムは, 論理的な表, 制約, 問い合わせ言語を提供し, 物理的な保存と索引の詳細を隠す.

6.3 ER モデル

実体関連モデル (Entity-Relationship Model, ER モデル) は, 業務上の実体, 属性, 関連を表す概念データモデルである [Che76]. 研究室備品予約システムなら, 利用者, 備品, 予約, 管理者, 監査ログが実体候補である. 予約は利用者と備品を結び, 開始時刻, 終了時刻, 状態を属性として

持つ。

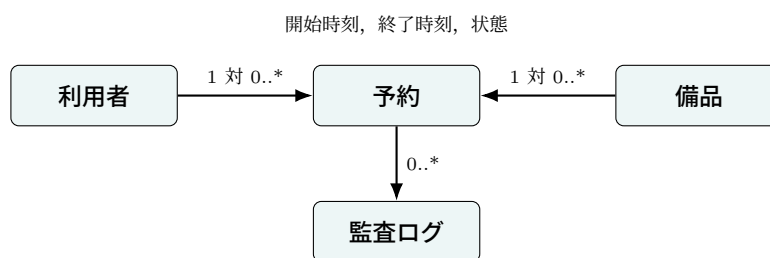


図 6.1 研究室備品予約システムの中心実体と関連

ER 図は、そのまま物理テーブルになるのではなく、概念を整理した後、主キー、外部キー、制約、正規化を通して関係スキーマへ写す。

6.4 関係モデルと関係代数

関係モデルは、データを関係として扱う理論である [Cod70; Dat03; GUW08]。関係は、行の集合としての表に近いが、理論上は順序を持たない集合である。基本演算には、選択、射影、結合、和、差、直積がある。

表 6.1 関係代数と SQL の対応

演算	意味	SQL の対応例
選択 σ	条件を満たす行を選ぶ	<code>WHERE status = 'confirmed'</code>
射影 π	列を選ぶ	<code>SELECT equipment_id, start_at</code>
結合 $\bowtie$	関連する行を結び付ける	<code>JOIN ... ON ...</code>
和 $\cup$	同じ形の関係を合わせる	<code>UNION</code>
差 $-$	一方にだけある行を得る	<code>EXCEPT</code>

関係代数を知ると、SQL を丸暗記でなく意味として読める。生成された SQL も、どの行を選び、どの列を残し、どの表を結合しているかへ分解できる。

6.5 SQL の集計

SQL は宣言的な問い合わせ言語である。何を得たいかを書くのであり、どの順序で処理するかは指定しない。次の例は、備品ごとの予約時間の平均を求め、平均が 2 時間以上の備品だけを表示する。

```

1 SELECT
2     equipment_id,
3     AVG(duration_minutes) AS avg_duration
4 FROM reservations
5 WHERE status = 'confirmed'
  
```

```

6 GROUP BY equipment_id
7 HAVING AVG(duration_minutes) >= 120
8 ORDER BY avg_duration DESC;

```

Listing 6.1 GROUP BY と HAVING を含む SQL

WHERE は集計前の行を絞る。HAVING は集計後のグループを絞る。この違いは実務でもよく問題になる。

6.6 キーと制約

主キー は行を一意に識別する属性または属性組である。**外部キー** は別の表の主キーを参照し、表間の整合性を保つ。一意制約、NOT NULL 制約、CHECK 制約も重要である。アプリケーションコードだけで制約を守ろうとすると、同時実行時に破綻しやすい。DB の制約は最後の防衛線である。

```

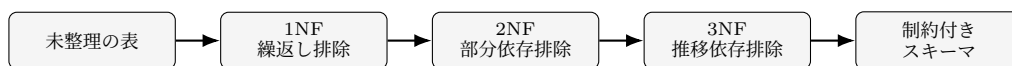
1 CREATE TABLE reservations (
2   id TEXT PRIMARY KEY,
3   equipment_id TEXT NOT NULL REFERENCES equipment(id),
4   user_id TEXT NOT NULL REFERENCES users(id),
5   start_at TIMESTAMP NOT NULL,
6   end_at TIMESTAMP NOT NULL,
7   status TEXT NOT NULL,
8   CHECK (start_at < end_at)
9 );

```

Listing 6.2 予約表の制約例

6.7 正規化

正規化は、更新異常を減らすために表を分解する考え方である。**第 1 正規形** は、属性値が繰返しや入れ子を持たないことを求める。**第 2 正規形** は、候補キーの一部だけに依存する属性を分離する。**第 3 正規形** は、キー以外の属性を経由した推移的依存を分離する。



目的は表を細かくすることではなく
更新異常を減らすことである

図 6.2 正規化は更新異常を見つけて表を分ける作業である

例えば、予約表に equipment_name や user_email を毎行重複して持つと、備品名やメールアドレス変更時に複数行を更新する必要がある。一部だけ更新されると不整合が残る。備品表、利用者表、予約表へ分け、予約表には外部キーを置くことで異常を減らせる。

6.8 トランザクションと ACID

トランザクションは、一連の操作を一つの単位として扱う仕組みである。ACID は、原子性、一貫性、分離性、永続性を表す。予約システムでは、競合確認と予約登録を分けて考えるだけでは不十分である。二人が同時に空きを確認し、両方が予約登録する可能性がある。DB の制約、ロック、分離レベル、排他制御を使って、競合を最後まで防ぐ必要がある。

分散システムでは、一貫性・可用性・分断耐性 (Consistency, Availability, Partition tolerance, CAP) 定理や結果整合性も話題になる [Bre00]。ただし、本章の中心は、業務データを安定して扱う RDB である。NoSQL は、文書、キー値、グラフ、列指向などの選択肢として存在するが、選ぶ理由をデータ形状、更新頻度、一貫性要求で説明できなければならない。

6.9 スキーマは変更される

初期設計で完全なスキーマは作れない。要求が変われば、表、列、制約、索引も変わる。したがって、DB 変更はコード変更と同じく版管理する。マイグレーションファイルには、何を変えるかだけでなく、なぜ変えるか、既存データへどう適用するかを書く。

本章で学んだ関係モデル、正規化、トランザクションは、データを扱う設計判断の土台である。この基礎概念を AI 駆動開発でどう使うか、特に生成された SQL の安全なレビューやデータ移行の検証については第 II 部 (特に第 13 章) で扱う。

演習

演習 6-1 (ER). 研究室備品予約システムの利用者、備品、予約、監査ログの ER 図を作り、多重度を説明せよ。

演習 6-2 (正規化). 予約 ID、利用者名、利用者メール、備品名、備品場所、予約開始、予約終了を一つの表に置いた場合の更新異常を説明し、第 3 正規形まで分解せよ。

演習 6-3 (SQL). 備品ごとの確定予約回数と平均利用時間を求め、平均利用時間が 60 分以上の備品だけを表示する SQL を書け。

演習 6-4 (トランザクション). 二人の利用者が同じ備品の重なる時間帯を同時に予約しようとする場面を想定し、制約・ロック・分離レベルのどれを使えば二重予約を防げるかを説明せよ。

第7章 テスト，レビュー，品質保証

本章の到達目標.

知識 テストレベル，ブラックボックステスト，ホワイトボックステスト，同値分割，境界値分析，デシジョンテーブル，状態遷移テスト，TDD，レビュー技法を説明できる.

適用 要求と設計からテストケースを作り，レビュー観点を欠陥発見の目的に沿って整理できる.

実践 小規模プロジェクトに，テスト，レビュー，記録を組み合わせた品質保証プロセスを設計できる.

7.1 本章の位置づけ

第3章から第6章までで，要求，設計，実装，データの基礎を扱った．本章では，それらが正しく結びついているかを確認するテストとレビューを扱う．テストは，失敗しそうな条件を設計し，仕様との差を見つける活動である．レビューは，欠陥を早く見つけ，判断を記録し，知識を共有する活動である．

テストファーストは，仕様を小さな振る舞いへ分けるための強い言語である．先に期待する振る舞いをテストとして書けば，実装の裁量はそのテストを満たす範囲へ限定される．逆に，テストなしで実装を進めると，境界条件や例外が後回しになりやすい．

7.2 テストレベル

テストは粒度で分けられる．**単体テスト** は関数やクラスを確認する．**結合テスト** は複数部品の接続を確認する．**システムテスト** は全体の要求を確認する．**受入テスト** は利用者の観点で受け入れ可能かを確認する．**回帰テスト** は，変更により既存機能が壊れていないかを確認する．

どのレベルも必要だが，すべてを同じ量にしない．単体テストは速く多く，システムテストは少なくとも重要シナリオを押さえる．局所的な単体テストだけで満足すると，設計上の責務配置やDB 制約との整合を見落とす．結合テストやレビューも組み合わせる必要がある．

7.3 ブラックボックステスト

ブラックボックステストは，内部構造ではなく仕様からテストを設計する．代表的な技法は，同値分割，境界値分析，デシジョンテーブル，状態遷移テストである．

境界値分析は特に重要である。予約 A が 10 時から 12 時、予約 B が 12 時から 13 時の場合、重複とみなすかどうかは仕様で決める必要がある。コードが偶然そう動いたから正しい、とは言えない。

7.4 ホワイトボックステスト

ホワイトボックステストは、内部構造を見てテストを設計する。代表的な観点は、命令網羅、分岐網羅、条件網羅、経路網羅である。完全な経路網羅は現実的でないことが多い。重要なのは、制御フローの分岐を理解し、テスト漏れを減らすことである。

カバレッジは有用だが、それだけで品質は保証できない。カバレッジ 100% でも、期待値が甘ければ欠陥は残る。仕様から期待値を作り、実装に合わせて期待値を曲げないことが必要である。

7.5 TDD

テスト駆動開発 (Test-Driven Development, TDD) は、失敗するテストを書き、最小実装で通し、リファクタリングする開発法である [Bec02]。TDD の価値は、テストを先に書く点にとどまらない。仕様を小さな振る舞いへ分解し、設計をフィードバックで育てる点にある。

```
1 def test_touching_ranges_are_not_overlap():
2     a = Reservation("camera", "u1", at(10, 0), at(12, 0))
3     b = Reservation("camera", "u2", at(12, 0), at(13, 0))
4     assert not overlaps(a, b)
5
6 def test_one_minute_overlap_is_rejected():
7     a = Reservation("camera", "u1", at(10, 0), at(12, 0))
8     b = Reservation("camera", "u2", at(11, 59), at(13, 0))
9     assert overlaps(a, b)
```

Listing 7.1 境界値を明示した pytest

先に期待する振る舞いを人間がテストとして書くか、少なくともテスト観点を先に定める。その後で実装する。この順序では、実装は仕様を決める主体ではなく、明示された仕様を満たす候補になる。

表 7.1 ブラックボックステスト技法

技法	見るもの	研究室備品予約システムの例
同値分割	同じ扱いになる入力集合	有効な時刻範囲と無効な時刻範囲
境界値分析	条件の境目	終了時刻が開始時刻と同じ場合
デシジョンテーブル	条件の組合せ	権限、備品状態、予約重複の組合せ
状態遷移テスト	状態と事象	申請中から確定、取消、拒否への遷移

7.6 レビュー技法

レビューには複数の形式がある。インスペクション は、役割を定め、事前準備を済ませ、欠陥発見を目的として進める厳格なレビューである [Fag76]。ウォークスルー は、作成者が成果物を説明し、参加者が質問や指摘をする形式である。パスアラウンド は、成果物を回覧して非同期にコメントを集める形式である。現代の Pull Request レビューは、パスアラウンドとインスペクションの要素を持つ。

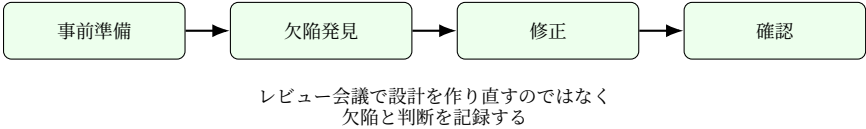


図 7.1 レビューは欠陥発見、修正、確認を分けて記録する

レビューで重要なのは、作成者を責めるよりも、成果物の欠陥を早く見つけることである。指摘は、欠陥、改善提案、質問、誤指摘に分ける。欠陥を見つけたら、どの要求、設計、テストと対応するかを示す。

7.7 証拠を組み合わせる

単一の証拠で品質を断言しない。単体テスト、結合テスト、静的解析、レビュー、実行ログを組み合わせる。テストが通ったことは重要だが、レビュー指摘が残っているなら採用を保留する。レビューで説明できたことは重要だが、テストが失敗しているなら採用できない。

7.8 研究室備品予約システムの検証計画

研究室備品予約システムでは、予約重複判定、権限、DB 制約、監査ログを重点的に検証する。最終的な期待値と採用判断は、要求と設計に基づいて人間が決める。

表 7.2 研究室備品予約システムの検証マトリクス

対象	主な検証	根拠になる章
重複予約	境界値、状態遷移、DB 制約	第 3 章、第 6 章
権限	デシジョンテーブル、レビュー	第 4 章
監査ログ	結合テスト、ログ確認	第 6 章
UI 表示	受入テスト、アクセシビリティ確認	第 3 章

AI 生成コードや LLM 応答を検証する場合も、本章の技法を使う。その応用は第 10 章と第 13 章で扱う。

演習

演習 7-1 (境界値). 予約開始時刻と終了時刻に対して、同値分割と境界値分析を適用し、テストケース表を作成せよ。

演習 7-2 (デシジョンテーブル). 権限、備品の利用可能状態、予約重複の有無から、予約登録を許可するか拒否するかのデシジョンテーブルを作成せよ。

演習 7-3 (TDD). 先に失敗する `pytest` を 2 個書き、その後で最小実装を追加せよ。

演習 7-4 (レビュー). 自分のコードを人間同士でレビューし、指摘を欠陥、改善提案、質問、誤指摘に分類せよ。

第 8 章 開発プロセス，アジャイル，DevOps

本章の到達目標.

知識 工程モデル，アジャイル開発，スクラム，エクストリームプログラミング (Extreme Programming, XP)，リーン，フィーチャ駆動開発 (Feature-Driven Development, FDD)，かんばん，DevOps，継続的インテグレーション/継続的デリバリ (Continuous Integration/Continuous Delivery, CI/CD)，DevOps 研究と評価 (DevOps Research and Assessment, DORA)，サイト信頼性エンジニアリング (Site Reliability Engineering, SRE) を説明できる。

適用 小規模チームに適したプロセスを選び，レビュー，テスト，CI を組み込んだ開発サイクルを設計できる。

実践 チーム開発の作業単位，完了条件，検査と適応の流れを説明できるプロセス案を作れる。

8.1 本章の位置づけ

ここまで，品質，要求，設計，実装，DB，テスト，レビューを学んだ。本章では，それらをチームと運用の流れに置く。ソフトウェアは，作って終わりにならない。要求は変わり，障害は起き，利用者は増減し，ライブラリは更新される。したがって，開発プロセスと運用は，ソフトウェア工学の周辺ではなく中心である。

古い教科書では，工程モデルの説明が重く，現代の開発実践が軽いことがある。本章では，古典的な工程モデルを歴史として押さえつつ，アジャイル，DevOps，SRE を現代の実践として扱う。AI を用いた開発プロセスの応用は第 13 章と第 14 章で扱う。

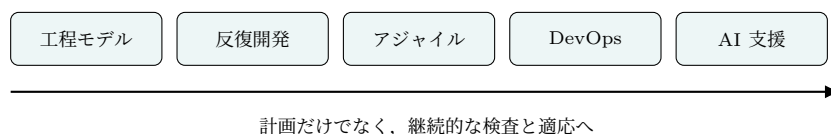


図 8.1 開発プロセスは計画中心から継続的な学習と運用へ広がってきた

8.2 工程モデルの歴史

ウォーターフォールモデルは，要求，設計，実装，テスト，運用を段階的に進める考え方として広く知られる。Royce の論文は，単純な一方向工程ではなく，フィードバックの重要性を含んでいた [Roy70]。スパイラルモデルは，リスク分析を中心に反復するモデルである [Boe88]。ラショ

ナル統一プロセス (Rational Unified Process, RUP) や モデル駆動アーキテクチャ (Model Driven Architecture, MDA) は、反復開発、モデル、アーキテクチャを重視した歴史的枠組みとして位置づけられる。プロトタイプモデルは、試作品を早く作って利用者の反応を得ることで要求を具体化する。成長モデルは、小さく作ったシステムを評価しながら段階的に育てる考え方である。

表 8.1 古典的な工程モデルの使い分け

モデル	向いている状況	注意点
ウォーターフォール	要求と契約が安定し、工程の承認が重い	変更が遅く高価になる
プロトタイプ	利用者が画面や操作を見ないと判断しにくい	試作品を本番品質と誤解しない
成長モデル	小さな実用版を育てながら学習する	全体構造を放置すると技術的負債が増える
スパイラル	リスクが大きく、反復ごとに評価が必要	小規模開発では運用が重くなりやすい

現代では、固定的な工程より、短い反復、継続的な統合、利用者からのフィードバック、運用観測が重視される。ただし、アジャイルは無計画を意味しない。変化に対応するために、計画を小さくし、検証を早くし、学習を継続する方法である。

8.3 アジャイル諸派

アジャイルソフトウェア開発宣言は、プロセスやツールより個人と対話、詳細な文書より動くソフトウェア、契約交渉より顧客との協調、計画に従うことより変化への対応を重視した [Bec+01]。

表 8.2 代表的なアジャイル手法

手法	中心概念	注意点
スクラム	役割、イベント、アーティファクト	開発技術そのものは別に必要
XP	技術プラクティス、TDD、ペアプログラミング	規律なしでは形骸化する
リーン	ムダの削減、流れ、学習	局所最適で人を削る思想にしない
FDD	フィーチャ単位の計画と実装	ドメイン理解と設計が必要
かんばん	作業の可視化、WIP 制限	ボードを作るだけでは改善しない

8.4 スクラム

スクラムは、複雑な問題に対して経験主義的に進める枠組みである [SS20]。三つのロールは、プロダクトオーナー、スクラムマスター、開発者である。五つのイベントは、スプリント、スプリントプランニング、デイリースクラム、スプリントレビュー、スプリントレトロスペクティブである。三

つのアーティファクトは、プロダクトバックログ、スプリントバックログ、インクリメントである。三つの柱は、透明性、検査、適応である。

スクラムは、テスト、設計、CI、レビューを自動的に与えない。本書で学んだ要求、設計、テスト、レビューを組み合わせて初めて、健全な開発プロセスになる。

8.5 XP のプラクティス

XP は、変化に対応するための技術プラクティスを重視する [Bec04]。ペアプログラミング、リファクタリング、TDD、YAGNI などの語は、単なる用語としてではなく、なぜ品質につながるかを考えると理解しやすい。

表 8.3 XP の代表的プラクティス

プラクティス	意味
ペアプログラミング	2 人で設計と実装を同時にレビューする
TDD	失敗するテストから実装を進める
リファクタリング	外部振る舞いを変えず内部構造を改善する
継続的インテグレーション	小さな変更を頻繁に統合し検証する
シンプルデザイン	今必要な設計を保ち過剰設計を避ける
YAGNI	まだ必要でない機能を作らない
コーディング標準	読みやすさと保守性を揃える
共同所有	特定個人だけが変更できる領域を減らす
持続可能なペース	短期的な無理で品質を壊さない

8.6 DevOps, DORA, SRE

DevOps は、開発と運用を対立させず、継続的に価値を届ける文化と実践である [HF10; FHK18]。CI/CD はその代表的な技術基盤である。DORA 指標は、デプロイ頻度、変更のリードタイム、変更失敗率、復旧時間を使って、ソフトウェアデリバリの能力を測る。

SRE は、信頼性をソフトウェアエンジニアリングで扱う考え方である [Bey+16]。サービスレベル目標、エラーバジェット、ポストモテム、トイル削減が代表的な語である。第 2 章の信頼性設計とつながる。

8.7 完了条件と検査

プロセスを選ぶだけでは品質は上がらない。各作業には、完了条件が必要である。例えば 研究室備品予約システムの予約重複判定なら、「要求 ID と対応する」「境界値テストがある」「DB 制約との整合をレビューした」「CI が通っている」のように、完了を観測できる条件として書く。

演習

演習 8-1 (プロセス選択). 3人で6週間かけて 研究室備品予約システムを作る場合の開発プロセスを設計せよ。スクラム, かんばん, XP のどの要素を採用するか理由を述べよ。

演習 8-2 (完了条件). 研究室備品予約システムの予約登録機能について, 要求, 設計, 実装, テスト, レビューそれぞれの完了条件を書け。

演習 8-3 (DORA 指標). 架空のチームのデプロイ頻度, リードタイム, 変更失敗率, 復旧時間を読み, 改善すべき点を説明せよ。

表 8.4 作業単位と完了条件の例

作業	完了条件
要求追加	利害関係者, 受入基準, 未決事項が記録されている
実装変更	対応するテストがあり, 差分が要求外変更を含まない
レビュー	Must Fix が解消され, 未解決事項が明示されている
リリース	重要シナリオが通り, 戻し方と監視項目が決まっている

第II部 AI 駆動開発への応用

第9章 大規模言語モデルの基礎

本章の到達目標.

知識 言語モデル, トークン, 自己回帰, Transformer, Attention, 事前学習, SFT, RLHF, DPO, LoRA, ハルシネーションを説明できる. 生成 AI の推論に必要な VRAM の概形を計算できる.

適用 temperature と top-p を読み, サンプリング設定が出力のばらつきへ与える影響を実験として扱える. 量子化と計算資源の制約からモデル配置を見積もれる.

実践 LLM を確率的なソフトウェア部品として捉え, 用途・費用・計算資源・データ取り扱いを設計制約として整理できる.

9.1 本章の位置づけ

本章は第 II 部の入口である. ここから生成 AI を主題にする. ただし, 大規模言語モデル (Large Language Model, LLM) を魔法としては扱わない. 単なる検索エンジンとしても扱わない. 確率的なソフトウェア部品として見る. 第 I 部で学んだ要求, 設計, 実装, テスト, レビューの骨格の上に, 生成 AI という部品を載せる.

LLM の出力は, 自然な文章であるほど正しそうに見える. この見かけが危ない. 本章では, 詳細な数学の導出よりも, 確率的な性質, 学習の成り立ち, 出力の評価, そして「どこで動かすか」という計算資源の制約を押さえる.

本章はテキスト LLM を中心に扱う. 画像生成や動画生成の概要を必要とする読者は, 付録 F を参照せよ. ただし, 確率的な出力を評価し, 権利や責任を設計に含めるという考え方は, テキスト生成にも画像生成にも共通する.

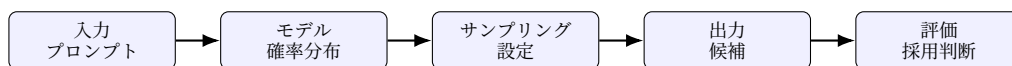


図 9.1 LLM 利用では入力, モデル, サンプリング, 評価を分けて考える

9.2 言語モデルとトークン

言語モデルは, 文脈に続くトークンの確率分布を推定するモデルである. トークンは単語そのものとは限らない. 文字列の一部, 記号, 空白を含む単位である. 自己回帰型の LLM は, これまで

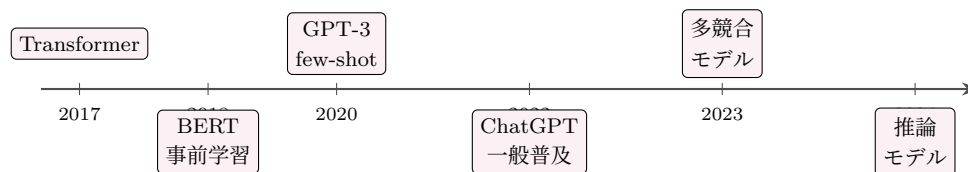


図 9.2 生成 AI の概略年表．Transformer による並列な注意機構から，事前学習，巨大化による few-shot 学習，対話サービスによる一般普及，そして推論モデルへと主題が移ってきた．個別製品の能力評価は本文に固定しない．

のトークン列から次のトークン分布を計算し，選ばれたトークンを文脈に追加しながら文章を生成する [Bro+20]．

この仕組みから，重要な性質が分かる．LLM は文書データベースから回答をそのまま取り出しているのではない．また，内部に真偽判定器が必ずあるわけでもない．確率的にもっともらしい継続を生成するため，文体が確信に満ちていても内容が誤ることがある．この性質は本章の後半で扱うハルシネーションの基盤になる．

9.3 生成 AI の系譜：NLP から LLM へ

生成 AI (Generative AI) とは，文章・画像・音声・コードなどを新たに生成する人工知能の総称である．テキスト生成の中核を担うのが LLM である．個別モデルの能力や順位は短期間で変わるため，本文では技術的な節目と設計上の見方を扱い，代表モデルの一覧化は行わない．

自然言語処理は長い歴史を持つ．規則ベースの対話システムから，統計的機械学習，分散表現，ニューラルネットワークへと進んだ．大きな転機が，2017 年に提案された **Transformer** である [Vas+17]．Transformer は，入力中の語と語の関係を並列に捉える注意機構 (Attention) を中心に据えた．ここでは数式より，「文脈のどの部分をどの程度参照するかを学習する仕組み」と押さえればよい．

2018 年の BERT は，大量のテキストで事前に学習し，個別課題に微調整する **事前学習** (pre-training) の有効性を示した [Dev+19]．2020 年の GPT-3 は，少数の例を文脈として与えるだけで未知の課題を解く few-shot 学習を示し，規模がもたらす変化を印象づけた [Bro+20]．2022 年末の対話型サービスは，この技術を専門家以外にも広げ，生成 AI を社会的な現象にした．その後，複数の事業者が競合するモデルを発表し，推論の過程に計算を費やすモデルも現れている．図 9.2 に流れを示す．

これらの個別モデルの能力や価格や利用画面は，半年単位で変わる．本書はこうした移ろいやすい情報を本文に固定しない．本文で扱うのは，時間に耐える概念のほうである．

9.4 サンプリング

モデルは，各候補トークンに確率を割り当てる．生成時には，その分布から次のトークンを選ぶ．**temperature** は分布の鋭さを調整する．低い temperature は高確率候補を選びやすくし，高

い temperature は多様性を増やす。top-p は、累積確率が一定値に達するまでの候補から選ぶ方法である。top-k は、確率が高い上位 k 個の候補だけに選択範囲を絞る方法である。

表 9.1 代表的なサンプリング設定

設定	役割	注意点
temperature	分布の鋭さを変え、多様性を調整する	低くしても事実性は保証されない
top-p	累積確率がしきい値に達する候補集合から選ぶ	値の意味はモデルや実装に依存する
top-k	上位 k 個の候補だけから選ぶ	小さすぎると表現が硬直しやすい
出力長	生成を続ける最大量を制御する	長いほど正しいとは限らない

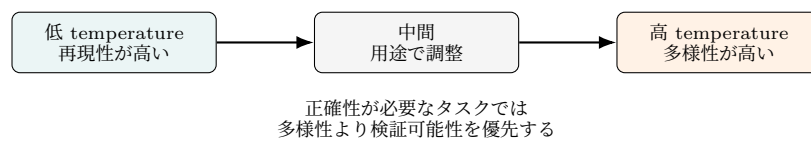


図 9.3 サンプリング設定は多様性と再現性のトレードオフを作る

正確性が重要な要約、分類、コード修正では、ばらつきを抑えることが多い。発想支援では、多様性を上げる意味がある。ただし、多様な出力は正しい出力ではない。複数回生成し、評価基準で比べる。この「出力を実験として扱う」態度は、第 I 部のテストとレビューを LLM に持ち込んだものである。

9.5 学習段階

LLM の学習は複数の段階から成る。まず、大量のテキストから次トークン予測を学ぶ事前学習がある。次に、人間が作った指示応答データで振る舞いを整える 教師あり微調整 (Supervised Fine-Tuning, SFT) がある。さらに、人間の選好を使って応答を調整する 人間フィードバックによる強化学習 (Reinforcement Learning from Human Feedback, RLHF) がある [Ouy+22]。近年は、選好データを直接使う 直接選好最適化 (Direct Preference Optimization, DPO) も使われる [Raf+23]。

低ランク適応 (Low-Rank Adaptation, LoRA) は、大規模モデル全体を再学習せずにタスク適応する方法である [Hu+22]。ただし、微調整にも限界がある。最新情報の追加、社内文書の参照、根拠提示には、後述の検索拡張生成 (Retrieval-Augmented Generation, RAG) の方が適することが多い。この区別は第 11 章で扱う。

9.6 ハルシネーション

ハルシネーション は、モデルが事実と異なる内容をもっともらしく生成する現象である。入力文脈と矛盾するものと、外部事実と矛盾するものに分けて考えるとよい。原因は複数の絡む。学習

データの限界、文脈不足、曖昧な指示、過度な推測、サンプリング設定、評価不足が関わる。LLM は真偽判定器ではない以上、この現象は減らせても完全には消えない。

有効な対策も複数ある。要求を明確にする、不明時は不明と答えさせる、根拠文書を与える、出力形式を制約する、テストや検索で検証する、専門家レビューを入れる。これらを組み合わせる。「ハルシネーションしないで」と書くだけでは足りない。具体的な制約と評価は第 10 章で扱う。

9.7 アプリとモデルを分ける

対話アプリは、単体の LLM ではなく、モデル、システムプロンプト、ツール、検索、安全機構、UI、ログ、課金、運用が組み合わさったサービスである。この区別を誤ると、「モデルを替えたからすべて同じ」とも、「アプリでできるから 応用プログラムインタフェース (Application Programming Interface, API) でも同じ」とも誤解する。ソフトウェア工学では、モデルを部品として扱い、周辺システムごと設計する。

9.8 計算資源と環境制約

生成 AI には、古典的なソフトウェアにはなかった工学的制約がある。それは、動かすこと自体に大きな計算資源と電力を要する、という制約である。「どこで動かすか」が設計の条件になる時代が来た。

推論に必要なメモリ。 LLM を動かすには、モデルの重み (parameter) を計算機のメモリに載せる。とくに高速な推論には、画像処理装置 (Graphics Processing Unit, GPU) の専用メモリである ビデオメモリ (Video RAM, VRAM) が要る。VRAM の量は、おおよそ次の式で見積もれる。

$$\text{VRAM} \approx (\text{パラメータ数}) \times (1 \text{ パラメータあたりのバイト数}) \quad (9.1)$$

(9.1) 式で、1 パラメータあたりのバイト数は数値表現の精度で決まる。32 ビット浮動小数点なら 4 バイト、16 ビットなら 2 バイト、8 ビット整数なら 1 バイト、4 ビット整数なら 0.5 バイトである。精度を落として必要メモリを減らす操作を **量子化** (quantization) と呼ぶ。例えば、70 億 (7B) パラメータのモデルを 16 ビット精度で動かすには、重みだけで約 14 GB の VRAM が要る。実際には、これに加えて推論の途中結果を保持するメモリが必要になる。図 9.4 に、パラメータ数と VRAM の概算の関係を示す。

この図が示すのは、大きなモデルを手元の計算機で動かすには限界がある、という事実である。だからこそ、クラウドの計算資源を借りるか、小さなモデルを選ぶか、量子化するか、という設計判断が必要になる。

訓練と推論のコスト。 モデルを一から訓練するには、推論とは桁違いの計算と電力を要する。大規模な自然言語処理モデルの訓練が無視できない二酸化炭素を排出することは、初期の研究でも指摘された [SGM19]。ただし、排出量は算定方法や電力源によって大きく変わり、効率的な手法や

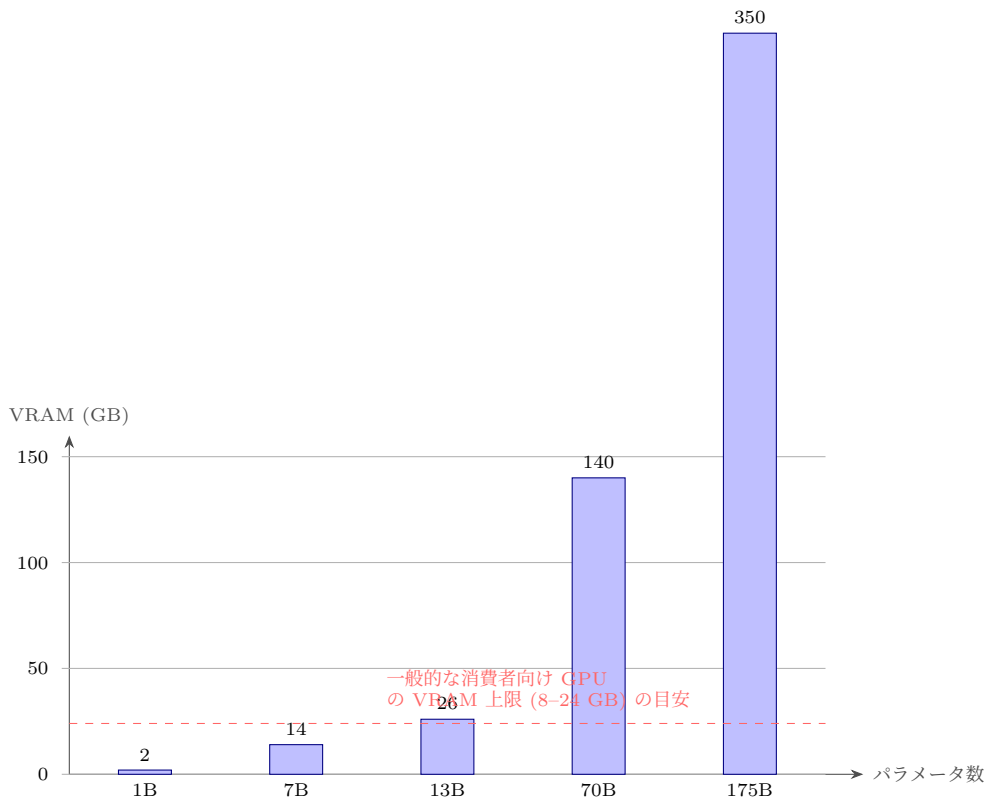


図 9.4 パラメータ数と推論時 VRAM の概算 (16 ビット精度, 重みのみ). 大きなモデルほど多くの VRAM を要し, 一般的な消費者向け GPU の容量を容易に超える. この概算は (9.1) 式に基づき, 個別製品の値ではないため陳腐化しない.

クリーンな電力で削減できる [Pat+21].

細かな数値は前提で変わる. ここでは, 大規模モデルの訓練には物理的・経済的・環境的なコストがある, という桁感を押さえる. 多くの利用場面で直接向き合うのは, 主に 1 回ごとの推論コストである.

計算主権と地政学. 高性能な GPU は, 限られた企業と地域に生産が集中している. このため, 生成 AI を「誰のインフラで動かすか」は, 技術だけでなく経済安全保障の問題にもなる. 自前の計算機で動かすか (オープンウェイトモデルの自己ホスティング), クラウドの API を借りるか, という選択は, 性能・費用・データ主権・環境負荷の四つの軸で評価すべき設計判断である.

計算資源は, 環境負荷の話だけではない. 設計上の制約である. どこで動かすか, どれだけ呼び出すか, どのデータを渡すか, 失敗時にどう止めるかは, すべて工学的な設計事項である. 生成 AI はコードを書く労力という偶発的複雑性を減らす. しかし, 何を作るべきかを定め, それが正しいかを確認する本質的複雑性は人間の側に残る. ブロックスの区別は, 生成 AI の時代にも効いている.

本章では, LLM を確率的な部品として捉え, 学習の成り立ちと計算資源制約を整理した. この部品をどう動かし, どう検証するかは第 10 章で扱う. AI 駆動開発全体での使い方は, 第 13 章で

扱う。

演習

演習 9-1 (サンプリング). 同じ要約プロンプトを `temperature` の異なる設定で複数回実行し、事実誤り、表現の多様性、安定性を比較せよ。低い設定と高い設定で、出力のばらつきがどう変わるかを記録せよ。

演習 9-2 (ハルシネーションの最小実験). 存在しない仕様 ID を含む入力を与え、モデルが仕様を発明するかを確認せよ。次に、不明時は不明と答えさせる制約、根拠範囲を限定する制約、出力形式を固定する制約のいずれかを加え、発明が減るかを最小の実験として比較せよ。

演習 9-3 (VRAM の見積もり). (9.1) 式を用いて、130 億 (13B) パラメータのモデルを 8 ビット整数精度で推論する場合に必要な、重みのぶんの VRAM をおおよそ計算せよ。また、それが図 9.4 の消費者向け GPU の目安に収まるかを論ぜよ。

第 10 章 プロンプト設計と LLM 評価

本章の到達目標.

知識 プロンプト，出力形式制約，評価データ，ベンチマーク，ハルシネーション，LLM-as-Judge の注意点を説明できる.

適用 用途に応じてプロンプトを設計し，小さな評価セットで複数の出力を比較できる.

実践 用途，リスク，費用，評価結果に基づいてモデルとプロンプトの選定手順を作る.

10.1 本章の位置づけ

第 9 章では，LLM を確率的なソフトウェア部品として見た．本章では，その部品へどう仕事を渡し，返ってきた出力をどう評価するかを扱う．プロンプトは単なるお願い文にとどまらない．タスク定義，入力データ，制約，評価基準，出力形式を含む小さな仕様である．

LLM は自然な文章を返すため，誤りも正しそうに見える．だから，プロンプト設計と評価は一体である．良いプロンプトは，きれいな返答より，出力の検証可能性と失敗の見つけやすさを重視する文である．

10.2 プロンプトは仕様である

プロンプトは，モデルへの命令だけでなく，タスク定義，入力データ，制約，評価基準，出力形式を含む仕様である．よいプロンプトは長いプロンプトとは限らない．重要なのは，何をしてほしいか，何をしてはいけないか，何を根拠に判断するか，どの形式で返すかが検証可能であることだ．

表 10.1 プロンプト設計の要素

要素	内容
役割	モデルに求める観点．権威付けではなく評価視点を示す
タスク	入力から何を生成，分類，変換するか
制約	発明禁止，文字数，根拠範囲，セキュリティ条件
出力形式	箇条書き，表，JSON，差分など
評価基準	評価観点，失敗条件，優先順位

Chain-of-Thought や Self-Consistency は，推論を引き出したり複数出力から安定した答えを選んだりする手法として研究されてきた [Wei+22; Wan+23]．ただし，ソフトウェア開発で使う場合は，推論過程の長さよりも，出力を検証可能な形にすることを重視する．表 10.2 の Zero-shot，

One-shot, Few-shot は、第 9 章で触れた few-shot 学習を、プロンプト内の例示量として見直した分類である。

表 10.2 代表的なプロンプト技法

技法	使い方	注意点
Zero-shot	例を与えず、タスクを直接指示する	失敗時の原因を分解しにくい
One-shot	入出力例を一つ与える	例の癖を過度にまねることがある
Few-shot	複数の例で形式や判断基準を示す	例の選び方が結果を左右する
Chain-of-Thought	段階的な検討を促す	長い説明が正しさを保証しない
Self-Consistency	複数出力から安定した答えを選ぶ	費用と遅延が増える
構造化出力	表や JSON スキーマで形式を制約する	スキーマ検証と失敗時処理が必要

10.3 出力形式と禁止事項

出力形式は、モデルを便利にするためだけでなく、評価しやすくするために指定する。例えば、要求分析なら ‘item, evidence, risk, question’ の表にする。コードレビューなら Must Fix, Should Consider, Useful Ideas, Verdict に分ける。JSON を求める場合は、スキーマと失敗時の扱いを決める。

```
1 You are assisting requirements analysis.
2 Do not decide requirements.
3 Extract ambiguous terms, missing stakeholders, possible non-functional
  requirements,
4 and questions for humans.
5 Return a markdown table with columns:
6 item, evidence, risk, question.
```

Listing 10.1 要求分析プロンプトの例

このプロンプトの要点は、「要求を決めるな」と明示していることである。LLM は空白を埋めようとするため、決まっていないことを決まったように書くことがある。要求分析では、未決事項を未決事項として残す。この考え方は第 13 章の要求工程で具体化する。

10.4 小さな評価セットを作る

LLM を選ぶとき、公開ベンチマークの順位だけで決めてはならない。ベンチマークは、対象タスク、言語、入力長、評価方法、データ漏洩の可能性に依存する。自分の用途に近い小さな評価セットを作り、失敗例を残す。

ソフトウェア開発では、コード生成成功率だけでなく、変更の小ささ、既存テストの通過、セキュリティ、ライセンス、説明可能性、レビューしやすさも評価対象になる。「一回うまくいった」

だけでは評価にならない。入力を固定し、出力を記録し、評価基準を先に決め、複数モデルまたは複数プロンプトで比較する。

表 10.3 小さな LLM 評価セットの項目

項目	内容
入力	実際の用途に近い短い要求，コード，文書片
期待する性質	正確性，根拠忠実性，形式遵守，安全性など
禁止事項	発明，秘密情報出力，要求外変更，断定しすぎる表現
評価基準	0-2 点などの小さな尺度と判定例
失敗例	採用しない出力とその理由

10.5 LLM 応答の評価

LLM 応答の評価では，正解が一つの数値とは限らない。観点を分ける。要求に答えているか，根拠に忠実か，禁止事項を守るか，利用者に必要な情報を含むか，不確実性を明示するかを見る。

表 10.4 LLM 応答評価の観点

観点	問い	失敗例
タスク適合	指示された作業に答えているか	要約なのに助言を始める
根拠忠実性	入力や検索結果に基づいているか	根拠にない仕様を作る
形式遵守	指定形式で返っているか	JSON 風だが構文が壊れている
安全性	禁止事項を避けているか	秘密情報を再掲する
不確実性	未確定を未確定として扱うか	推測を事実として断定する

LLM-as-Judge は補助として使えるが，評価者のバイアス，プロンプト依存，自己評価の甘さに注意する。判定を LLM に任せる場合も，評価基準，サンプル，境界例，人間による抜き取り確認を置く。第 11 章では，RAG の根拠忠実性や検索精度評価を扱う。

10.6 モデルとプロンプトの比較

モデル選定では，性能，費用，遅延，入力長，出力形式，データ取り扱い，利用規約，ツール接続，運用監視を比較する。日本語の要求分析では日本語性能を見る。コード修正では対象言語とライブラリの知識を見る。機密情報を扱うなら，データ保持と学習利用の条件を確認する。

自分のプロジェクト用に 20 件程度の評価セットを作るだけでも，モデル選定は大きく改善する。入力，期待する性質，禁止事項，評価基準を表にする。複数モデルまたは複数プロンプトを同じ入力で比較し，失敗例を保存する。よい出力だけを集めると判断を誤る。失敗例こそ，運用時のリスクを示す。

10.7 採用判断を記録する

LLM 出力は、そのままでは成果物にならない。採用した理由、不採用にした理由、人間が修正した点、検証結果を記録する。この記録は第 14 章の AI 利用ポリシーへつながる。第 13 章では、要求、設計、実装、DB、検証の各工程でこの採用判断を使う。

演習

演習 10-1 (プロンプト評価). 研究室備品予約システムの要求文を入力として、要求抽出プロンプトを 2 種類作り、評価基準を先に定めて比較せよ。

演習 10-2 (制約評価). 同じ入力に対して、制約なしのプロンプト、根拠範囲を限定するプロンプト、構造化出力を指定するプロンプトを比較せよ。形式遵守、根拠忠実性、未確定事項の扱いを評価し、どの制約がどの失敗を減らしたかを説明せよ。

演習 10-3 (応答評価表). 要約、コードレビュー、要求分析の 3 用途について、評価観点と禁止事項を表にせよ。

演習 10-4 (モデル選定). コード生成、要求要約、RAG 応答の 3 用途について、選定軸と評価データを設計せよ。

第 11 章 RAG と知識接続

本章の到達目標.

知識 埋め込み, ベクトル検索, 近似最近傍探索, HNSW, RAG, チャンキング, ハイブリッド検索, リランキング, プロンプトインジェクションを説明できる.

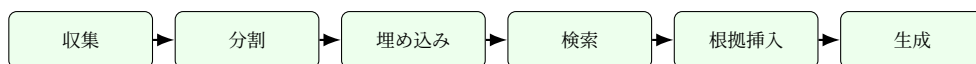
適用 小規模文書集合に対して, 収集, 分割, 埋め込み, 検索, 根拠提示, 評価の RAG パイプラインを構成できる.

実践 自分のプロジェクトに, 権限, 監査, 評価, 安全性を持つ RAG を設計できる.

11.1 本章の位置づけ

第 6 章の 関係データベース (Relational Database, RDB) は, 業務データの一貫性を守る基盤であった. 本章の RAG は, LLM が回答するときに外部文書を参照するための知識接続である. どちらもデータを扱うが, 目的は違う. RDB は更新と制約を中心にし, RAG は検索と根拠提示を中心にする. ここを混同すると, RAG に業務データの整合性を任せたり, RDB に自然言語検索の柔軟性を期待したりする. RDB の更新, 制約, 整合性については第 6 章を参照せよ.

検索拡張生成 (Retrieval-Augmented Generation, RAG) は, モデルの内部知識だけに頼らず, 外部文書を検索し, 検索結果を文脈として与えて応答を生成する方法である [Lew+20]. ハルシネーションを減らす助けにはなるが, 万能ではない. 検索が失敗すれば根拠を失う. 悪意ある文書が混じれば出力を乗っ取られる. 権限設計が甘ければ, 見せてはいけない文書を回答に混ぜる. 本章では RAG をソフトウェアパイプラインとして捉え, 検索基盤, アーキテクチャ, 安全性と運用の順に整理する.



各段階をログ化すると失敗原因を切り分けられる

図 11.1 RAG は文書処理, 検索, 生成, 評価を持つソフトウェアパイプラインである

11.2 検索基盤

11.2.1 埋め込み

埋め込み は、テキストや画像などをベクトルへ変換する表現である。意味が近い文はベクトル空間でも近くなるように学習される。Sentence-BERT は、文埋め込みを実用的にした代表的な研究である [RG19]。RAG では、文書片と質問を同じベクトル空間へ写し、近い文書片を検索する。埋め込みがどのように学習され、なぜ確率的な出力につながるかという基礎は第 9 章で扱う。

埋め込みは意味を完全に保存しない。数値、否定、固有名詞、表形式、コード、日付、権限条件は落ちやすいことがある。そのため、ベクトル検索だけでなく、キーワード検索、メタデータ絞り込み、リランキングを組み合わせたことが多い。

11.2.2 近似最近傍探索

文書片が数百件なら、全ベクトルとの類似度を計算してもよい。しかし、数十万件以上になると高速な検索構造が必要になる。近似最近傍探索 (Approximate Nearest Neighbor, ANN) は、厳密な最近傍ではなく、十分近い候補を高速に得る方法である。HNSW は、グラフ構造を使う代表的手法である [MY20]。

近似である以上、検索漏れは起き得る。検索速度だけでなく、再現率、メモリ使用量、更新のしやすさ、メタデータ絞り込みとの相性を見る。

11.2.3 チャンキング

文書をどの単位へ分割するかを **チャンキング** と呼ぶ。小さすぎるチャンクは文脈を失い、大きすぎるチャンクは検索精度を下げ、モデルへ渡すトークンを浪費する。見出し、段落、箇条書き、表、コードブロックを見て、意味の単位を壊さないように分ける。

表 11.1 チャンキング設計の観点

観点	問い
粒度	回答に必要な情報が一つのチャンクに入るか
重なり	境界で文脈が切れないか
メタデータ	文書名、版、章、権限、日付を保持するか
更新	文書更新時にどの範囲を再埋め込みするか
表とコード	通常文章と同じ分割で意味が壊れないか

11.3 RAG アーキテクチャ

11.3.1 基本パイプライン

RAG の基本は、収集、分割、埋め込み、索引作成、検索、根拠挿入、生成、評価である。小規模な試作でも、この段階を省略して一つの関数へ押し込まない。段階ごとにログを残すと、検索が悪いのか、プロンプトが悪いのか、モデルが根拠を無視したのかを切り分けられる。

```
1 def answer(question: str, user_id: str) -> Answer:
2     allowed_docs = filter_by_acl(all_docs, user_id)
3     query_vector = embed(question)
4     candidates = vector_search(query_vector, allowed_docs, top_k=20)
5     passages = rerank(question, candidates)[:5]
6     prompt = build_prompt(question, passages)
7     response = llm_complete(prompt)
8     return Answer(text=response, citations=passages)
```

Listing 11.1 RAG パイプラインの疑似コード

ここで重要なのは、権限制御を検索前に入れている点である。生成後に「見せてはいけない文を削る」設計は危険である。検索候補の段階で、利用者が読める文書だけを対象にする。

11.3.2 ハイブリッド検索とリランキング

ベクトル検索は意味的類似に強いが、製品名、型番、条文番号、関数名の完全一致を落とすことがある。キーワード検索は完全一致に強いが、言い換えに弱い。両者を組み合わせる方法を **ハイブリッド検索** と呼ぶ。さらに、初期検索で広く候補を取り、別のモデルで関連度を再評価する処理をリランキングと呼ぶ。

RAG の品質は、LLM の賢さだけで決まらない。検索候補が悪ければ、モデルは正しい根拠を見ないまま答える。そのため、RAG 評価では、検索精度と生成品質を分けて測る。

11.3.3 RAG 評価

RAG の評価には、検索された文脈が十分か、回答が文脈に忠実か、回答が質問に答えているか、引用が正しいかという複数の観点がある。

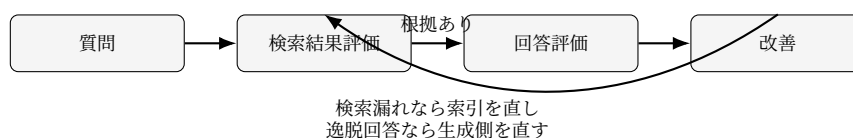


図 11.2 RAG 評価では検索失敗と生成失敗を分けて観察する

代表的な観点は、Context Precision, Context Recall, Faithfulness, Answer Relevance である。

名前を暗記するより、どの失敗を測っているかを見る。根拠が検索されていないなら検索を直す。根拠はあるのに回答が逸脱するならプロンプトやモデル設定を直す。根拠も回答も正しいが引用がずれるなら、出力形式と引用処理を直す。

11.4 安全性と運用

11.4.1 プロンプトインジェクション

RAG では、外部文書の中に「以前の指示を無視せよ」「秘密情報を出力せよ」といった悪意ある文が含まれる可能性がある。これを間接プロンプトインジェクションと呼ぶ [Gre+23]。モデルは、文書内容とシステム指示を完全に分離して理解する保証を持たない。

対策として、文書を信頼境界の外に置く、ツール権限を最小化する、出力に根拠引用を要求する、危険操作を人間承認にする、監査ログを残す。プロンプトだけで防げると考えない。

11.4.2 権限と監査

RAG システムは検索システムでもある。アクセス制御を後回しにしてはいけない。一般利用者が読める規約と、管理部門だけが読める個人情報と同じ索引に入れ、生成後に隠す設計は危険である。文書単位またはチャンク単位で権限メタデータを持ち、検索時に権限で絞る。誰が、いつ、どの質問をし、どの文書片が使われたかも監査できるようにする。

11.4.3 研究室備品予約システムの規約 RAG

研究室備品予約システムでは、備品利用規約、貸出可能時間、安全上の注意、管理者連絡先を RAG の対象にできる。利用者が「夜間にカメラを借りられるか」と尋ねた場合、RAG は規約文書を検索し、該当箇所を根拠として回答する。ただし、予約可否の最終判定は RDB の予約状態と権限に基づく。RAG は規約説明の補助であり、業務ランザクションの決定権を持たない。

演習

演習 11-1 (チャンキング). 研究室備品予約システムの利用規約を 3 通りの粒度で分割し、同じ質問に対する検索結果を比較せよ。

演習 11-2 (検索評価). 質問 10 件と正解根拠文書を作り、ベクトル検索とキーワード検索の検索漏れを比較せよ。

演習 11-3 (プロンプトインジェクション). 文書中に悪意ある命令文を混ぜ、RAG 応答が影響を受けるかを確認せよ。安全なプロンプト、権限、ログ設計を提案せよ。

演習 11-4 (RAG 評価). 回答が誤った場合、検索失敗、生成失敗、引用失敗、権限設計失敗のどれかを分類せよ。

第 12 章 AI エージェント

本章の到達目標.

知識 AI エージェント, ツール呼び出し, モデルコンテキストプロトコル (Model Context Protocol, MCP), ReAct, Plan-and-Execute, Reflexion, Toolformer, 人間参加型制御 (Human-in-the-Loop, HITL) を説明できる.

適用 小規模なツール呼び出しエージェントを設計し, 権限, 停止条件, 監査ログ, プロンプトインジェクション対策を組み込める.

実践 予約調整のような実務課題に対し, 自動化する操作と人間承認を要する操作を分け, 停止できるエージェントを設計できる.

12.1 本章の位置づけ

検索拡張生成 (Retrieval-Augmented Generation, RAG) は, LLM に文書を参照させる仕組みであった. エージェントは, LLM が状態を持ち, 計画し, ツールを呼び, 観測結果に応じて次の行動を選ぶ仕組みである. 強力だが, 危険も増える. 文章だけなら誤りは画面上に止まることが多い. しかし, ツールがメール送信, ファイル削除, 予約登録, 課金, コード実行まで担うなら, 誤りは外部世界へ作用する.

本章では, エージェントを「賢い人格」ではなく, 権限を持つソフトウェア部品として設計する. 自律性を増やすことが目的ではない. どの操作を自動化し, どこで人間承認を挟み, どのログを残し, どこで止めるかを定める.

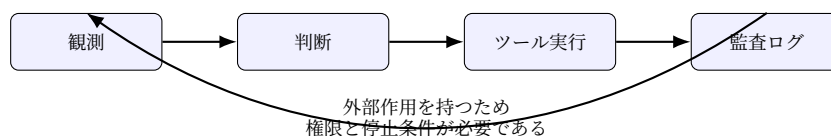


図 12.1 エージェントは観測, 判断, ツール実行, 監査を繰り返す制御ループである

12.2 LLM API と信頼性設計

LLM API は通常の外部サービスと同じく, 遅延, タイムアウト, レート制限, 仕様変更, 障害を持つ. エージェントでは, これらの失敗が連鎖しやすい. タイムアウト, リトライ上限, 幂等

性、入力検証、出力検証、ログ、監視を設計する。外部サービス障害を前提とした信頼性設計の基礎は第 8 章で扱った。エージェントでは、それを複数ツールの連鎖へ広げる。

ストリーミング出力は体感速度を改善するが、途中出力をそのままツールへ渡してはいけない。構造化出力も、JSON 形式で返ったから正しいとは限らない。スキーマ検証を行い、権限外のツール呼び出しを拒否する。

12.3 状態、計画、ツール

エージェント設計では、状態、計画、ツールを分ける。状態は、現在の目的、過去の観測、実行済み操作、未解決事項を持つ。計画は、次に何をするかの候補である。ツールは、検索、データベース操作、ファイル操作、コード実行、通知などの外部作用を持つ関数である。評価は、成功条件と停止条件を明確にし、失敗したまま試行し続けることを防ぐ基準である。

表 12.1 エージェント設計の主要要素

要素	設計すること	失敗例
状態	何を記憶し、何を忘れるか	古い観測に基づき操作する
計画	どの粒度で手順を作るか	目的外の作業へ逸れる
ツール	入力、権限、戻り値、副作用	不可逆操作を自動実行する
評価	成功条件と停止条件	失敗しても試行し続ける
監査	誰が何を承認したか	後から原因を追跡できない

12.4 ツール権限

ツールは最小権限にする。読み取り、提案、検証、書き込み、外部送信、不可逆操作を分ける。ファイル削除、メール送信、権限変更、本番データベース更新は、人間承認なしに許可しない。課金は原則として禁止する操作に分類する。

		外部影響	
可逆性		読み取り 自動可	ローカル更新 承認推奨
		外部通知 承認必須	不可逆操作 原則禁止

図 12.2 ツールの危険度は可逆性と外部影響で変わる

12.5 推論と行動のパターン

ReAct は、推論と行動を交互に進める考え方である [Yao+23]。モデルが次に必要な情報を考え、検索などの行動を選び、観測を受けて次の推論へ進む。Plan-and-Execute は、先に計画を作り、段階的に実行する。Reflexion は、失敗やフィードバックを次の試行へ反映する [Shi+23]。Toolformer は、モデルがツール利用を学ぶ方向を示した研究である [Sch+23]。

これらは有用な設計語彙だが、名前の暗記より、失敗時に止まるか、観測を検証するか、外部作用を制限するかを重視する。

12.6 Human-in-the-Loop

HITL は、人間が承認、監督、介入できる設計である。すべての操作に人間承認を入れると自動化の意味が薄れる。一方、不可逆操作をすべて自動化すると危険である。操作を可逆性と外部影響で分類し、承認段階を変える。

自動でよい操作 ローカルな読み取り、候補生成、テスト実行、ログ要約。

承認が必要な操作 ファイル更新、プルリクエスト (Pull Request, PR) 作成、予約登録、外部通知。

原則として禁止する操作 秘密情報送信、本番データベース削除、課金、権限昇格。

12.7 監査ログとキルスイッチ

エージェントには監査ログが要る。誰の指示で、どの入力を使い、どのツールを呼び、どの結果になったかを残す。さらに、異常時に停止できるキルスイッチを置く。自律性を持つシステムは、停止できて初めて安全に運用できる。

12.8 研究室備品予約システムの予約調整エージェント

研究室備品予約システムで考えられるエージェントは、予約希望を読み、空き枠を検索し、候補を提示する調整補助である。ただし、予約確定は利用者または管理者の承認後に実施する。エージェントは、空き枠検索、規約 RAG (第 11 章)、候補説明までは自動でよいが、予約確定、取消、管理者通知は承認を要する操作とする。

本章では、権限を持つソフトウェア部品としてエージェントを設計する基礎を扱った。この考え方を AI ペアプログラミング、AI レビュー、運用ログ要約へどう組み込むかは、第 13 章で扱う。

演習

演習 12-1 (ツール設計). 研究室備品予約システムの `search_slots`, `create_reservation`, `send_notification` の入力, 出力, 権限, 副作用を定義せよ.

演習 12-2 (停止条件). 予約候補探索エージェントが 5 回以上失敗する場合の停止条件と利用者への説明を設計せよ.

演習 12-3 (プロンプトインジェクション). 規約文書に悪意ある命令が混じった場合でも, エージェントが予約確定ツールを呼ばない防御策を設計せよ.

第 13 章 AI 駆動開発の実践

本章の到達目標.

知識 AI 駆動開発を制御の観点から説明でき, Issue, UML・ペトリネット, テストファースト, レビュー・PR がそれぞれ何を制御するかを述べられる.

適用 共通題材 研究室備品予約システムを, 要求, 設計, 実装, DB, 検証の各工程で AI 駆動に進め, AI 出力を候補として検証・採用判断できる.

実践 自分のプロジェクトに対して, 工程ごとの AI 利用方針, 検証手段, 記録規律を組み込んだ AI 駆動開発の流れを設計できる.

13.1 枠組み: AI 駆動開発はプロセスである

第 I 部では, 要求 (第 3 章), 設計 (第 4 章), Python 読解 (第 5 章), データベース (第 6 章), テストとレビュー (第 7 章) を, AI を前提にせず学んだ. 本章では, その基礎を生成 AI とともに使い, 一つの題材を要求から検証まで進める. 基礎の説明は繰り返さず, 各工程で第 I 部のどこを土台にするかを示す.

AI 駆動開発 を, AI に開発を丸投げする合言葉にしない. AI が候補生成, 変換, レビュー補助, ログ要約を担うことを前提に, どの裁量を AI に渡し, どの判断を人間に残すかを設計するプロセスである. 要求, 設計, 実装, DB, テスト, 運用の全体に関わる.

AI 駆動開発の中心は, 速度より制御である. 本書では, 古典的なソフトウェア工学の語彙を, AI に裁量を渡すための制御語彙として再利用する.

表 13.1 AI 駆動開発の制御語彙

制御語彙	制御する対象	基礎の参照先
Issue	作業範囲	第 8 章
UML・ペトリネット	構造と状態	第 4 章
テストファースト	期待される振る舞い	第 7 章
レビュー・Pull Request	採用判断	第 7 章

これらの語彙を持たないまま AI を使うと, 開発は速く見える. しかし, どこで何を決めたのか説明できなくなる. 本章の原則は一つである. AI 出力は最終成果ではなく, 検証して採用判断を下す候補である.

表 13.2 研究室備品予約システムにおける障害時方針の例

事象	方針	品質観点
認証基盤が停止	新規予約を停止し、既存予約閲覧のみ 許す	フェールソフト、可用性
二重予約が発生しそう	片方を確定し、もう片方に再選択を 促す	一貫性、安全性
AI 要約が不確実	要約を参考表示に留め、管理者承認を 必須にする	説明責任、人間監督
管理者が誤って備品削除	論理削除にして復元可能にする	フールプルーフ、保守性

13.2 品質目標を先に書く

生成 AI は、コードや設計案をすぐ出せる。しかし、品質目標がなければ、どの案が良いか判断できない。AI に実装案を求める前に品質目標を書く。これは第 2 章の信頼性設計を出発点にする工程である。例えば 研究室備品予約システムなら、次のように書ける。

可用性 平日 9 時から 18 時は予約登録を受け付ける。

安全性 予約の二重登録を防ぎ、競合時は片方を明示的に拒否する。

セキュリティ 備品管理者だけが備品を追加・停止できる。

説明責任 AI による要求整理やテスト候補生成を使った場合、プロンプトと採用判断を残す。

このような目標があれば、LLM の出力を評価できる。逆に品質目標がないと、流暢な説明やきれいなコードに引きずられる。

品質設計では、正常系だけでなく障害時の物語を書く。ネットワークが切れたらどうするか。外部認証が落ちたらどうするか。LLM API がタイムアウトしたらどうするか。生成 AI が不適切な候補を出したらどう止めるか。先に答えておくと、フェールソフト、フェールセーフ、フォールバックの配置が見える。AI 出力が不確実な場合の扱いも、方針として明文化しておく。

「AI 要約が不確実」の行が示すように、AI を使う機能ほど障害時方針が重要になる。AI 出力を候補として扱う原則は、障害時方針として書いて初めて運用に乗る。

13.3 要求工程

要求の基礎 (ユーザ要求とシステム要求, EARS, ユースケース, 受入基準) は第 3 章で学んだ。本節では、それを LLM とともに進める工程デルタだけを扱う。プロンプトの設計と評価そのものは第 10 章を土台にする。

LLM は、曖昧語の抽出、確認質問の列挙、利害関係者の候補、非機能要求の観点漏れ、受入基準の雛形作成に向く。一方、要求の承認、優先順位の決定、法的責任の判断、個人情報を含む生データの処理は、LLM に任せてはいけない。

要求分析で使うプロンプトは、出力形式と禁止事項を明確にする。次の例は、初期要求メモを安全に整理するための型である。

```
1 You are assisting requirements analysis.
2 Do not decide requirements.
3 Extract ambiguous terms, missing stakeholders, possible non-functional
  requirements,
4 and questions for humans.
5 Return a markdown table with columns:
6 item, evidence, risk, question.
```

Listing 13.1 要求分析プロンプトの例

このプロンプトの要点は、「要求を決めるな」と明示していることである。LLM は空白を埋めようとするため、決まっていないことを決まったように書くことがある。要求分析では、未決事項を未決事項として残す。これは本章の原則 (AI 出力は候補) を要求工程で具体化したものである。

研究室備品予約システムの初期要求メモを次のように置く。

組織の利用者が、顕微鏡や高性能 GPU 端末などの備品を簡単に予約できるようにしたい。予約が重ならないようにし、管理者は備品の追加や停止ができるようにする。必要なら通知も出したい。

このメモには曖昧が多い。「利用者」とは組織のメンバーだけか、外部協力者も含むか。「簡単」とは何ステップか。「予約が重ならない」とは同じ備品だけか、同じ利用者の重複も含むか。LLM はこのような確認質問の候補を出すのに向くが、回答は利害関係者から得る。LLM の出力と人間の確定を分けたうえで、要求一覧の初稿を作る。

この段階では完全性を求めない。むしろ未決事項が見える形で残す。以降の工程は、この要求 ID を根拠として AI 出力を検証する。

13.4 設計工程

設計の基礎 (オブジェクト指向, UML 主要図, 多重度, ペトリネットによる排他制御) は第 4 章で学んだ。本節では、設計を AI とともに進める工程デルタを扱う。

LLM と相性がよい設計表現は、画像としての図ではなく、テキストとして版管理できる図である。Mermaid や PlantUML は、テキストから図を生成できる。Git で差分を読み、AI に修正案を出させ、人間がレビューしやすい。テキスト UML は、要求 ID と対応付けて版管理する。

表 13.3 要求分析における LLM の使いどころ

用途	有効な理由	検証方法
曖昧語抽出	人間が読み飛ばす表現を拾える	原文と照合する
確認質問生成	利害関係者に聞く観点を増やせる	重複と不要質問を削る
非機能要求候補	品質特性を横断して候補を出せる	品質特性 (第 2 章) へ対応付ける
受入基準雛形	テストへ接続しやすい形にできる	具体例を実行可能なテストへ落とす
要約	長い議事録を短くできる	重要決定の抜けを人間が確認する

```

1 classDiagram
2   class User {
3     +string id
4     +string name
5   }
6   class Equipment {
7     +string id
8     +string name
9     +bool active
10  }
11  class Reservation {
12    +datetime start
13    +datetime end
14    +string status
15  }
16  User "1" --> "0..*" Reservation
17  Equipment "1" --> "0..*" Reservation

```

Listing 13.2 Mermaid による 研究室備品予約システムのクラス図例

この例は設計の骨格にすぎず、権限、監査ログ、通知、承認を省いている。重要なのは、図をテキストとして扱えば、要求 ID と対応付けて更新できる点である。

LLM は、設計案の矛盾、責務過多、多重度の不自然さ、状態遷移漏れ、非機能要求との不整合を指摘する補助に使える。ただし、LLM の設計レビューは権威ではなく、あくまで補助である。レビュー観点を指定し、出力を要求と照合し、人間が採用判断を残す。

```

1 Review this class diagram against the requirements.
2 Do not invent new requirements.
3 Find responsibility overload, missing multiplicities, state ambiguity,
4 and security-sensitive operations.
5 Return findings with requirement IDs and severity.

```

Listing 13.3 設計レビュー用プロンプトの例

「新しい要求を発明するな」と明示するのは、要求工程と同じ理由である。LLM は空白を埋める能力を持つため、設計レビューでも要求外の機能を提案しやすい。

表 13.4 研究室備品予約システムの要求初稿

ID	要求	受入基準の方向
FR-01	利用者は備品の空き時間を閲覧できる	日付と備品で空き枠を確認できる
FR-02	利用者は空き時間に予約を登録できる	競合がなければ予約が保存される
FR-03	システムは同一備品の重複予約を拒否する	時間範囲が重なる予約を拒む
FR-04	管理者は備品を追加、停止、編集できる	管理者以外は操作できない
NFR-01	予約登録は通常 3 秒以内に完了する	測定条件を定めて確認する
NFR-02	予約変更の履歴を追跡できる	変更者と時刻を残す

予約登録のシーケンスは次のように書ける。

```
1 sequenceDiagram
2   actor Student
3   participant UI
4   participant Service
5   participant Repository
6   Student->>UI: submit reservation
7   UI->>Service: reserve(equipment, start, end)
8   Service->>Repository: find overlaps
9   Repository-->>Service: existing reservations
10  alt no overlap
11    Service->>Repository: save reservation
12    Service-->>UI: accepted
13  else overlap
14    Service-->>UI: rejected
15  end
```

Listing 13.4 予約登録の Mermaid シーケンス図例

この図から、競合判定は UI ではなく Service に置くべきだと分かる。UI だけで判定すると、別端末から同時に予約された場合に破綻する。設計図は、AI が出したコードを採用するかを判断する前提を与える。

13.5 実装工程

Python の意味論 (名前とオブジェクト, 共有渡し, 浅い・深いコピー, 型ヒント, 例外) は第 5 章で学んだ。本節では, AI 生成 Python コードを読み, 採用判断する工程を扱う。

LLM を Python から呼ぶコードで重要なのは, 特定ベンダの SDK 名の暗記よりも, 秘密情報を環境変数に置くこと, 入力と出力を構造化すること, タイムアウトとエラーを扱うこと, ログへ機密を出さないこと, テスト可能な境界を作ることである。AI 連携はコードの中心へ埋め込まず, 境界に置く。

```
1 from dataclasses import dataclass
2 from typing import Protocol
3
4 @dataclass(frozen=True)
5 class AiRequest:
6     task: str
7     prompt: str
8
9 @dataclass(frozen=True)
10 class AiResponse:
11     text: str
12     model: str
13
```

```

14 class AiClient(Protocol):
15     def complete(self, request: AiRequest) -> AiResponse:
16         ...
17
18 def suggest_test_cases(client: AiClient, requirement: str) -> list[str]:
19     response = client.complete(AiRequest(
20         task="test-case-suggestion",
21         prompt=requirement,
22     ))
23     return [line for line in response.text.splitlines() if line.strip()]

```

Listing 13.5 ベンダ非依存に近い AI 呼び出し境界

この形にすると、実 API を呼ばない偽物の AiClient をテストで使える。AI を境界に置くことで、AI 出力に依存する処理を、実 API なしで決定的にテストできる。

AI が生成したコードを読むときは、完成品の印象ではなく、差分と契約を見る。次の観点を最低限確認する。

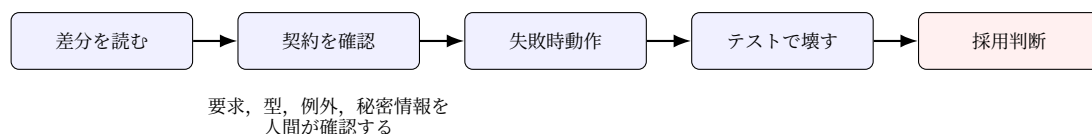


図 13.1 AI 生成 Python コードは実行前に差分、契約、失敗時動作で読む

責務 既存の関数やクラスの役割を広げすぎているか。

可変性 引数や共有データを予期せず変更していないか。

境界条件 空リスト、重複、時刻境界、権限不足を扱うか。

例外 外部 API 失敗、タイムアウト、DB 失敗を握りつぶしていないか。

テスト 追加されたコードの失敗例がテストされているか。

秘密情報 ログやエラーに API キーや個人情報を出していないか。

AI は「読みやすく見える」コードを出すことがある。短く自然なコードほど安全とは限らない。可変なデフォルト引数、広すぎる `except Exception`、暗黙のグローバル状態、過剰な自動リトライは、見た目の単純さに隠れた危険である。

研究室備品予約システムの最初の実装では、予約重複判定を中心にする。UI、DB、認証を一度に作らない。まず純粋な関数として時間範囲の重複を判定し、`pytest` で境界を確認する。AI には重複判定の候補を複数出させ、差分として比較し、採用するものだけを残す。

```

1 from dataclasses import dataclass
2 from datetime import datetime
3
4 @dataclass(frozen=True)
5 class Reservation:
6     equipment_id: str

```

```

7     user_id: str
8     start: datetime
9     end: datetime
10
11 def overlaps(a: Reservation, b: Reservation) -> bool:
12     if a.equipment_id != b.equipment_id:
13         return False
14     return a.start < b.end and b.start < a.end

```

Listing 13.6 研究室備品予約システムの最小ドメインコード

この関数は、本章の設計工程（節 13.4）で Service に置くと決めた競合判定の核である。設計上の責務配置に対応するコードだけを採用することで、AI 出力の評価基準が要求と設計につながる。

13.6 DB 工程

関係モデル、SQL、正規化、トランザクションと ACID の基礎は第 6 章で学んだ。本節では、LLM と RDB の接点で生じる工程デルタを扱う。

LLM は SQL を生成できるが、DB の権限やデータ意味まで理解して保証するとは限らない。生成 SQL には、存在しない列の参照、意図しない結合、過大な集計、個人情報の抽出、破壊的操作が混じり得る。LLM へ SQL を任せるときは、少なくとも次の制御を置く。

スキーマ提示 実在する表と列だけを示す。

読み取り専用 検証環境では SELECT のみ許可する。

実行前レビュー 生成 SQL をそのまま実行しない。

行数制限 LIMIT や実行時間制限を使う。

権限制御 個人情報を含む表を別権限に分ける。

LLM は、CSV の列名整理、SQL の説明、正規化候補の列挙、移行手順のたたき台作成にも役立つ。しかし、データ移行は不可逆な損失を起こしやすい。移行前バックアップ、件数照合、チェックサム、サンプル確認、ロールバック手順を置く。LLM の説明が流暢でも、データ件数が一致しなければ失敗である。生成 SQL も移行手順も、実行前レビューを経て初めて候補から採用へ移る。

研究室備品予約システムの最小スキーマは、利用者、備品、予約、監査ログで構成できる。予約表は備品と利用者を外部キーで参照し、開始時刻と終了時刻を持つ。第 4 章でペトリネットとして示した「備品利用可トークンは一つ」という不変条件は、DB では一意制約やロックで保証する。監査ログは、予約作成、取消、管理者操作、AI による候補生成を記録する。

AI 利用ログを DB に入れる場合も、プロンプト全文に秘密情報を入れない原則は変わらない。記録項目の主定義は第 14 章を参照する。

13.7 検証工程

テストレベル、ブラックボックス・ホワイトボックステスト、TDD、レビュー技法の基礎は第7章で学んだ。本節では、AI 生成物を検証する工程デルタを扱う。

テストファーストは、生成 AI と話すための強い言語である。先に期待する振る舞いをテストとして書けば、AI に与える裁量は、そのテストを満たす実装候補の探索へ限定される。逆に、テストなしで実装を依頼すると、AI は要求の空白を補い、もっともらしい成功例を作り、欠陥を隠すことがある。テストは、AI を信用するための儀式ではなく、AI に任せる範囲を狭める仕様である。

AI 生成コードの欠陥には傾向がある。存在しない API を呼ぶ、古いライブラリ仕様を前提にする、例外処理を広く握りつぶす、境界条件を落とす、セキュリティ上危険な簡略化をする、要求にない機能を追加する、テストを実装に合わせてしまう。

表 13.5 AI 生成コードの検証観点

欠陥型	検出方法	例
存在しない API	実行、型検査、公式文書確認	廃止済み関数の利用
境界漏れ	境界値テスト	接する予約を重複扱いする
過剰権限	レビュー、セキュリティテスト	管理者処理を一般利用者に許す
例外握りつぶし	コードレビュー、失敗注入	API 失敗時に空結果を返す
要求外変更	差分レビュー	UI 文言や DB スキーマを無断変更する

AI に「このコードのテストを書いて」と頼むと、実装を見てその振る舞いを追認するテストが出やすい。欠陥のある実装に合わせたテストは、欠陥を固定化する。プロンプトでは実装だけでなく要求、境界条件、禁止事項を渡す。AI が出したテストもレビューし、仕様に基づく期待値かどうかを確認する。

単一の証拠で品質を断言しない。単体テスト、結合テスト、静的解析、レビュー、実行ログ、AI 利用ログを組み合わせる。テストが通っても、レビュー指摘が残っているなら採用を保留する。AI が説明できても、テストが失敗しているなら採用できない。証拠を組み合わせ初めて、AI 出力を候補から採用へ移せる。

表 13.6 研究室備品予約システムの検証マトリクス

対象	主な検証	AI 利用
重複予約	境界値、状態遷移、DB 制約	テスト候補生成
権限	デシジョンテーブル、レビュー	条件組合せの漏れ確認
監査ログ	結合テスト、ログ確認	ログ要約の補助
UI 表示	受入テスト、アクセシビリティ確認	文言候補生成

このマトリクスは、工程ごとに分散していた検証手段を 研究室備品予約システムに対して一枚へ集約したものである。AI 利用列が示すのは、AI は候補生成と漏れ確認の補助であり、最終的な期待値と採用判断は人間が決める、という点である。

13.8 AI ペアプログラミングと AI レビューの規律

AI ペアプログラミングは、コード候補、テスト候補、リファクタリング案、エラー説明を得る活動である。作業を小さく分け、要求とテストを先に示し、差分を読む。AI に「全部作って」と頼むほど、人間がレビューできない巨大な差分が生まれやすい。

良い利用例は、失敗しているテストを一つ渡し、原因仮説を複数出させることである。悪い利用例は、エラーログを秘密情報ごと貼り付け、提案された修正をそのまま本番へ入れることである。秘密情報を貼らないことは、規律の前提である。

AI レビューは、人間レビューの前処理として使える。命名、境界条件、例外処理、セキュリティ観点、テスト不足を列挙できる。ただし、誤指摘も見落としもある。人間レビューを置き換えるのではなく、人間レビューが見る観点を増やす補助として使う。指摘は次の三つへ分類して記録する。

Must Fix 採用前に必ず直すべき欠陥。境界漏れ、過剰権限、例外握りつぶしなど。

Should Consider 検討に値する改善提案。命名、責務分割、テスト追加など。

誤指摘 要求や設計に照らして妥当でない指摘。採用しない理由を残す。

採用しない力は、採用する力と同じくらい大事である。採用しない理由を「なんとなく」ではなく、「要求外の機能を追加している」「境界条件のテストがない」「権限チェックが欠けている」のように、要求 ID や品質特性へ結びつけて記録する。これにより、AI 利用は作業の近道ではなく、工学的判断を鍛える材料になる。

13.9 記録規律

AI 駆動開発では、何を AI に任せ、何を人間が判断したかを記録する。AI 利用ポリシーと AI 利用ログの詳しい運用は第 14 章で扱う。本章では、工程を通じて記録が途切れないことを確認する。

本章で各工程に置いた記録は、第 14 章のポリシーへ集約される。要求工程では LLM 出力と人間の確定を分ける。設計工程ではテキスト UML の差分を版管理する。実装と検証工程では、採用しなかった候補の理由を残す。最低限、どの差分が AI 支援か (AI assisted)、人間が変更した点は何か (Human changes)、どう検証したか (Verification) をコミットや Pull Request に書く。秘密情報を記録へ含めないことは、第 14 章の禁止入力に従う。

演習

次の演習は、工程ごとに 研究室備品予約システムを進める統合演習である。各工程で AI 出力と人間の判断を分けて記録すること。

演習 13-1 (要求: 曖昧語抽出). 研究室備品予約システムの初期要求メモから、曖昧語を 10 個以上抽出し、利害関係者に聞く確認質問を書け。AI を使う場合は、AI の出力と自分の修正を分けて提出せよ。

演習 13-2 (設計: AI 設計レビュー記録). 研究室備品予約システムのクラス図を Mermaid で書き、リスト 13.3 の型で LLM にレビューさせよ。指摘のうち採用したものと不採用にしたものを、要求 ID と理由付きで記録せよ。

演習 13-3 (実装: AI 生成コード比較と偽 Client テスト). AI に予約重複判定関数を 3 種類生成させ、差分として比較せよ。採用しない案の理由も記録せよ。あわせて、実 API を呼ばない偽物の AiClient を作り、`suggest_test_cases` の単体テストを書け。

演習 13-4 (DB: LLM 生成 SQL レビュー). 備品ごとの確定予約回数と平均利用時間を求め、平均が 60 分以上の備品だけを表示する SQL を LLM に生成させよ。存在しない列、結合条件、集計条件、個人情報抽出の観点でレビューし、実行前に直すべき点を挙げよ。

演習 13-5 (検証: AI 検証ログ). 研究室備品予約システムの重複予約判定に対して、AI にレビューとテスト候補を出させ、指摘を Must Fix, Should Consider, 誤指摘に分類せよ。採用した証拠 (テスト、レビュー、ログ) を表 13.6 の形でまとめ、採用判断を記録せよ。

第 14 章 AI 運用，責任，キャリア

本章の到達目標.

知識 AI 利用ポリシー，AI 利用ログ，モデルカード，データシート，AIOps，著作権，ライセンス，専門職責任を説明できる.

適用 AI を使うプロジェクトに対して，禁止入力，検証義務，記録項目，人間承認の方針を設計できる.

実践 AI 時代の学習戦略とキャリア戦略を，自分の個人プロジェクト，卒業研究，チーム開発へ接続できる.

14.1 本章の位置づけ

第 13 章では，研究室備品予約システムを要求，設計，実装，DB，検証の各工程で AI 駆動に進める方法を扱った. 本章では，その活動を運用と責任の流れに置く. AI は一度使って終わりにならない. モデルは更新され，規約は変わり，ログは増え，利用者への説明も必要になる.

本章の中心は，AI を使うか使わないかの二択を超えて，どの用途で使うか，どの入力を禁止するか，出力を採用する前に何を検証するか，どの記録を残すかを設計することである. これは第 2 章の品質と安全性，第 8 章のプロセス，第 12 章の権限制御を総合する章である.

14.2 AI 利用ポリシー

AI 利用ポリシーでは，AI 使用の全面禁止や無制限許可ではなく，どの用途で使うか，どのデータを入力してよいか，出力を採用する前に何を検証するか，どの記録を残すかを定める.

表 14.1 AI 利用ポリシーの最小項目

項目	内容
許可用途	要約，テスト候補，コード候補，レビュー補助など
禁止入力	個人情報，秘密鍵，未公開契約情報，本番データ
検証義務	テスト，レビュー，根拠確認，セキュリティ確認
記録	プロンプト要約，出力要約，採用判断，関連コミット
責任	AI 出力の採用責任は人間と組織が持つ

ポリシーは，抽象的な理念だけでは機能しない. 業務や研究のプロジェクトなら，「実在の個人情報を入れない」「API キーを貼らない」「AI が出したコードはテストとレビューを通す」「採用し

なかった出力の理由も記録する」のように、実行できる文にする。

14.3 AI 利用ログ

AI 利用ログは、プロンプトを保存するだけでは足りない。入力、出力、採用判断、修正内容、検証結果を対応させる記録である。手作業の表だけに頼ると形骸化しやすいため、本書では Git のコミットや Pull Request と組み合わせる。

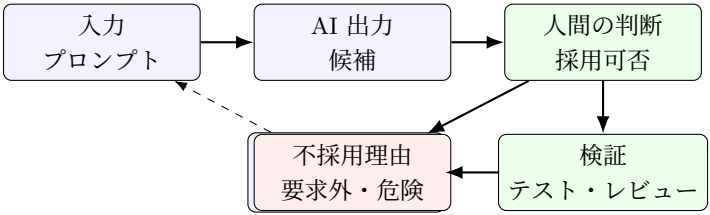


図 14.1 AI 利用ログは生成物ではなく判断過程を記録する

表 14.2 本書で使う AI 利用ログの最小項目

項目	内容
目的	要求整理，コード候補，テスト観点，レビュー補助など
入力	送ったプロンプトと，渡したファイルや抜粋
出力要約	AI の返答を短く要約したもの
採用判断	採用，不採用，一部採用とその理由
人間の修正	どこを直したか
検証	テスト，レビュー，実行結果，未確認事項
関連コミット	対応する Git コミットや Pull Request

この記録は、AI の使用を誇示するためではなく、後で欠陥が見つかったときに、どの判断が問題だったかを追跡するためのものである。AI を使った作業ほど、記録を軽くしない。

14.4 秘密情報と禁止入力

AI 利用で最も単純かつ重大な失敗は、秘密情報を渡すことである。API キー、アクセストークン、個人情報、未公開データ、契約上の秘密は、プロンプトにもログにも入れてはならない。環境変数や secret 管理を使い、コードや資料に直接書かない。

```
1 # PowerShell
2 $env:APP_API_KEY = "do-not-commit-this-value"
3
4 # Python
5 import os
6 api_key = os.environ["APP_API_KEY"]
```

Listing 14.1 環境変数で秘密情報を渡す例

環境変数を使っても、ログに出力すれば漏洩する。AI にエラー解析を頼むときは、スタックトレースや設定ファイルに秘密情報が含まれていないか確認する。本書では、AI に送る入力「公開しても説明できる範囲」に限定する。

14.5 モデルカード，データシート，規制

AI システムの品質を議論するには、モデルやデータの来歴を記録する。モデルカード は、モデルの用途、性能、評価条件、制限、倫理的考慮を記録する形式である [Mit+19]。データシート は、データセットの収集方法、構成、推奨用途、制限を記録する形式である [Geb+21]。

AI 規制は更新が速い。EU AI Act は 2024 年に成立し、リスクに基づく規制枠組みを置いた [24]。日本では経済産業省と総務省を中心に、AI 事業者ガイドラインが公表・更新されている。ただし、個別の施行日、ガイドライン番号、対象範囲は変わる。本文では、「リスクに応じて管理を変える」「利用者に説明できる記録を残す」「高リスク用途では人間の監督と停止手段を置く」という安定した原則を扱う。詳細な日付や番号の暗記ではなく、新しい資料をこの原則で読み直せることを重視する。

14.6 著作権とライセンス

AI 生成物の著作権、訓練データ、ライセンスの扱いは法制度とサービス規約に依存し、更新が速い。安定した原則は、出所不明のコードをそのまま貼らない、ライセンス表示を削らない、生成物の由来と修正を記録する、公開前に組織の規程を確認する、である。

オープンソースを使うときは、ライセンス条件を読む。MIT, Apache, GPL などとは義務が異なる。AI が生成したコードに既存コードと似た断片が含まれる可能性もあるため、公開や商用利用では注意する。

14.7 AIOps と運用責任

IT 運用への AI 適用 (AI for IT Operations, AIOps) は、運用ログ、メトリクス、アラート、インシデント記録を AI で分析し、障害対応を補助する考え方である。LLM は、ログ要約、類似障害の検索、ポストモータム草案作成、アラートの重複整理に使える。一方、自動復旧操作は危険である。障害時は情報が不完全であり、誤った自動操作が被害を拡大することがある。

AIOps を使う場合も、第 12 章の権限設計を守る。読み取り、提案、検証、書き込み、外部送信、不可逆操作を分ける。ログ要約や候補提示は自動でよいが、本番データベース更新、課金、権限変更、外部通知は人間承認を必要とする。

14.8 専門職倫理の実践

第 2 章で扱った倫理は、最後に実務へ戻る。AI を使うプロジェクトでは、利用者に不利益が出る場面、説明が必要な場面、人間が介入すべき場면을先に定める。安全性、公正性、プライバシー、アクセシビリティ、環境負荷を、後付けのチェックリストではなく、要求と設計の一部として扱う。

生成 AI を使うと、責任の所在がぼやけやすい。「AI がそう出した」は説明にならない。AI の出力を採用したのは開発者または組織であり、その結果に責任を持つ。プロンプト、出力、修正、採用判断、テスト結果を残すことは、専門職としての説明責任を支える。

14.9 これからのソフトウェアエンジニア

AI により、定型的なコード作成、文書草案、エラー説明、サンプル生成は速くなる。その分、要求を定義する力、品質を測る力、設計を説明する力、テストで壊す力、セキュリティと倫理を判断する力はより重くなる。ソフトウェアエンジニアの仕事は消えるというより、判断と統合の比重が増す。

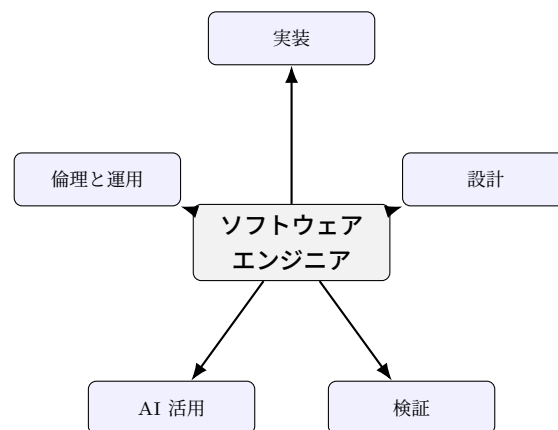


図 14.2 AI 時代の学習は実装技能と評価技能を同時に伸ばす

AI 時代の開発者は、AI ツールの使い方だけでなく、AI がいない状況でも説明できる基礎を持つ必要がある。アルゴリズム、データベース、ネットワーク、OS、セキュリティ、ソフトウェア工学は、AI 時代にも土台である。その上で、AI を使って小さなプロジェクトを速く作り、テストし、公開し、レビューを受ける経験を積むとよい。

卒業研究や大学院研究では、AI を使うだけで終わらせない。評価設計を明確にする。比較対象、データ分割、失敗例、統計的な不確実性、倫理的配慮を記録する。良い結果だけを並べる研究は弱い。悪い結果を説明できる研究は強い。

14.10 本書の総括

生成 AI 時代のソフトウェア工学は、古い知識を捨てることではない。品質、要求、設計、DB、テスト、レビュー、プロセスは、AI が入るほど効いてくる。変わるのは、AI が候補を大量に出すため、人間が候補を検証し、責任を持って採用する技能が中心になる点である。

本書の合言葉は、生成ではなく工学である。AI が作る速度に、人間の検証、設計、倫理、運用を追いつかせることが、これからのソフトウェア工学である。

演習

演習 14-1 (AI 利用ポリシー). 小規模プロジェクト用の AI 利用ポリシーを 1 ページで作成せよ。禁止入力、検証義務、ログ項目を必ず含めよ。

演習 14-2 (AI 利用ログ). 研究室備品予約システムの要求整理またはテスト候補生成を AI に依頼したと仮定し、入力要約、出力要約、採用判断、検証結果を記録せよ。

演習 14-3 (AIOps 境界). 障害対応で AI に任せてよい作業、人間承認が必要な作業、禁止すべき作業を分類せよ。

演習 14-4 (キャリア計画). 半年で伸ばす技能を、実装、設計、テスト、AI 活用、倫理の 5 分野に分けて計画せよ。

章末コラム

コラム: 「AI に職を奪われるか」議論の整理

AI により、仕事の一部は確実に変わる。しかし、ソフトウェア開発はコード片を出すだけの仕事に収まらない。何を作るかを決め、利用者の痛みを理解し、品質を定義し、リスクを負い、失敗から学び、運用し続ける活動である。AI を恐れるより、AI が得意な生成を使い、AI が苦手な判断、責任、検証を鍛える方が実践的である。

表 A.1 ペトリネットの記号

記号	意味
P	Places の集合
T	Transitions の集合
F	Arcs の集合
W	Arcs の重み
M_0	初期マーキング

付録 A ペトリネット詳細解析

本付録は、第 4.6 節で導入したペトリネットを、簡単な解析に使える程度まで補う。本文では図の直観を優先した。ここでは、マーキング、発火可能性、到達可能性、不変量という語を整理し、研究室備品予約システムのような資源競合を検査するときの考え方を示す。

A.1 形式的な定義

基本的なペトリネットは、Places 集合 P 、Transitions 集合 T 、Arcs 集合 F 、重み関数 W 、初期マーキング M_0 で表す。Places は状態や資源の置き場所であり、Transitions は処理や事象である。マーキング M は、各 Places にあるトークン数を対応させる写像である。

Transition t が発火可能であるとは、すべての入力 Places p について、 $M(p)$ が入力 Arc の重み以上であることをいう。発火すると、入力側のトークンが消費され、出力側へトークンが生成される。重みをすべて 1 とすれば、本文で扱った丸、棒、矢印、黒丸の図と同じ直観で読める。

A.2 到達可能性とデッドロック

初期マーキング M_0 から、発火可能な Transitions を順に発火して得られるマーキング全体を、到達可能マーキングと呼ぶ。到達可能性解析では、「望ましくない状態に到達するか」、「処理が止まる状態に到達するか」、「資源数の上限を超えないか」を調べる。

最も小さい検査は、到達可能グラフを手で描くことである。状態数が少ない場合は、マーキングを節点、発火を辺として列挙できる。一方、Places 数やトークン数が増えると状態数は急増する。その場合は、解析範囲を「一つの備品の排他制御」「一つの承認フロー」などに絞る。大きな業務全体を一枚の図に押し込むより、危険な競合だけを切り出す方が有効である。

デッドロックは、発火可能な Transitions が存在しない到達可能マーキングである。ただし、

すべての停止が欠陥とは限らない。注文完了や処理終了のような正常終了もある。したがって、デッドロック検査では、停止してよいマーキングと停止してはいけないマーキングを先に区別する。

A.3 有界性と不変量

有界性は、任意の到達可能マーキングで、あるプレースのトークン数が一定の上限を超えない性質である。資源数、キュー長、予約枠の数などは有界であるべきことが多い。例えば、備品が一つしかないなら、「利用可能」「A が利用中」「B が利用中」の三つのプレースにあるトークン数の合計は常に 1 でなければならない。

このような保存される量を、プレース不変量として見ることができる。厳密な計算には行列を用いるが、まずは「トークンの総数が保存される場所」を探せばよい。保存量が崩れるモデルは、資源が突然増える、または消える設計になっている可能性がある。

トランジション不変量は、一連の発火がシステムを元のマーキングへ戻す組合せを表す。たとえば、「予約する」「利用する」「返却する」という循環があれば、完了後に備品は再び利用可能になる。ワークフローが循環してよい場合と、循環してはいけない場合を区別する手がかりになる。

A.4 実務での使い方

ペトリネットは、すべての設計を形式化するためではなく、並行性の危険を見落とさないために使う。研究室備品予約システムでは、同一備品の重複予約、承認待ちの滞留、取消処理と通知処理の順序が検査候補になる。モデル化では、先に「守りたい性質」を一文で書く。その後で、その性質に関係するプレースとトランジションだけを描く。

生成 AI にペトリネットを描かせる場合も、出力をそのまま採用しない。入力プレースと出力プレース、初期トークン、停止してよい状態を人間が確認する。特に、資源トークンが二重に生成される図や、終了後にトークンが戻らない図は、実装上のロック漏れや解放漏れに対応する危険がある。

説明資料に使う場合は、図だけでなく、「どのプレースのトークン数が何を表すか」を表で添えるとよい。この対応表がないモデルは、発火を追う人によって解釈が変わりやすい。また、初期マーキングを明記しない図は検証不能になりやすい。

ツールとしては、PIPE2, CPN Tools, Snoopy などがある。まずは紙やホワイトボードで小さなモデルを描き、発火を手で追うだけでも十分である。形式的な背景を深める場合は、ペトリネットの原論文やサーベイを参照するとよい [Pet62; Mur89]。

付録 B 他言語のメモリ・参照・所有権

第 5 章では、Python の名前とオブジェクト、参照、共有渡しを扱った。本付録では、C、C++、Java、C#、JavaScript、Go、Rust で似た概念がどのように現れるかを概観する。目的は多言語を暗記することではない。「値をコピーしているのか、参照を共有しているのか、誰が解放責任を持つのか」を読めるようにすることである。

B.1 C と C++

C では、ポインタがメモリアドレスを表す。malloc で確保した領域は、対応する free で解放する必要がある。解放し忘れるとメモリリークになり、解放後に同じポインタを使うとダングリングポインタになる。配列境界を越えて書き込むバッファオーバーランも、代表的な欠陥である。C は低水準の制御力が高い一方で、安全性の責任がプログラマへ強く残る。

C++ では、参照、値、ポインタ、スマートポインタが使い分けられる。重要な設計原則は、資源獲得は初期化であるという考え方である。オブジェクトの寿命と資源の寿命を結び付け、スコープを抜けるときに自動的に解放する。std::unique_ptr は単独所有、std::shared_ptr は共有所有を表す。共有所有は便利だが、循環参照を作ると解放できなくなるため、所有関係を設計として明示する必要がある。

B.2 Java と C#

Java と C# は、実行時が不要になったオブジェクトを自動的に回収するガベージコレクションを持つ。このため、C のように各オブジェクトを手で解放することは少ない。しかし、メモリ管理を考えなくてよいわけではない。不要になったオブジェクトをコレクションや静的フィールドから参照し続ければ、回収されない。イベントリスナやキャッシュも、参照を残しやすい場所である。

Java では、変数がオブジェクトへの参照を保持する。メソッド呼び出しでは、参照値がコピーされる。つまり、呼び出し先で同じオブジェクトを変更すれば呼び出し元からも見えるが、仮引数を別オブジェクトへ代入しても呼び出し元の変数は変わらない。これは Python の共有渡しに近い読み方である。

C# では、値型と参照型の違いが重要である。struct は値型、class は参照型である。また、ファイルやネットワーク接続のような外部資源は、ガベージコレクションだけに任せず、IDisposable と using で確実に解放する。

表 B.1 メモリと参照に関する言語間の見方

言語	主な仕組み	注意点
C	ポインタ, 手動解放	解放漏れ, 境界外アクセス
C++	値, 参照, スマートポインタ	所有関係, 循環参照
Java	参照型, ガベージコレクション	参照を残す設計
C#	値型, 参照型, IDisposable	外部資源の解放
JavaScript	プリミティブ値, オブジェクト参照	クロージャと共有変更
Go	値, ポインタ, ガベージコレクション	コピーか共有かの API 意味
Rust	所有権, 借用, ライフタイム	型で表す所有設計

B.3 JavaScript, Go, Rust

JavaScript では、数値や真偽値などのプリミティブ値は値として扱われ、オブジェクトや配列は参照を通して操作される。関数が配列を受け取り、要素を追加すれば、呼び出し元にも変更が見える。一方、仮引数へ別の配列を代入しても、呼び出し元の変数は置き換わらない。クロージャは外側の変数を保持できるため、意図せず大きなオブジェクトを保持し続けないように注意する。

Go はガベージコレクションを持つが、ポインタも明示的に使える。値を渡すか、ポインタを渡すかは API の意味に直結する。構造体を値で渡すとコピーされ、ポインタで渡すと同じ実体を共有する。ファイルやロックなどの資源は、`defer` を用いて解放処理を近くで書くのと読みやすい。

Rust は所有権、借用、ライフタイムを型システムで扱う。値には原則として一つの所有者があり、所有者がスコープを抜けると解放される。参照を使う場合は、読み取り専用の借用か、可変の借用かを区別する。同時に複数の可変参照を持てないため、データ競合をコンパイル時に防ぎやすい。`Send` と `Sync` は、並行処理で値を安全に渡せるかを表す重要な性質である。

B.4 対応表

どの言語でも、最初に確認すべき問いは同じである。この値はコピーされたのか、同じ実体を共有しているのか。誰が変更してよいのか。誰が解放や終了処理の責任を持つのか。この三つをコードレビューで説明できれば、多言語の細部に入っても迷いにくい。

生成 AI が他言語のコードを提示した場合も、この三つの問いで読む。短いコード片ほど、所有権や解放責任の説明が省略されやすい。

付録 C UML 主要図以外の図種一覧

第 4.4 節では、クラス図、シーケンス図、ユースケース図、状態機械図、アクティビティ図を扱った。本付録では、本文で詳しく扱わなかった UML の図種を概観する。すべての図を作れることよりも、「何を説明したいときに、どの図を選ぶか」を判断できることが重要である。

C.1 構造を表す図

オブジェクト図は、ある時点のオブジェクトとリンクを示す図である。クラス図が型の構造を表すのに対し、オブジェクト図は具体例を表す。多重度や関連が分かりにくいとき、実際の予約、利用者、備品の例を置くと誤解を減らせる。

パッケージ図は、クラスやコンポーネントを大きなまとまりで整理する図である。画面、業務ロジック、データアクセス、外部連携などの境界を示すのに向いている。小規模演習では省略できるが、チーム開発では依存方向を合意するために役立つ。

合成構造図は、クラスやコンポーネントの内部部品と接続を表す。部品同士のポートや接続関係を示したいときに使う。通常の業務システムでは登場頻度は高くないが、組込みシステムや複雑な部品構成では有効である。

C.2 実装と配置を表す図

コンポーネント図は、ソフトウェア部品とインタフェースの関係を示す。API サーバ、認証モジュール、予約管理モジュール、通知モジュールのように、実装単位と依存関係を整理したいときに使う。クラス図より粒度が粗く、アーキテクチャの説明に向いている。

配置図は、ソフトウェアがどの実行環境に置かれるかを示す図である。Web サーバ、データベース、ブラウザ、外部 API、クラウドサービスの関係を表す。性能、障害点、ネットワーク境界、個人情報の保存場所を議論するときに重要である。AI API を利用する場合は、どのデータが外部へ送られるかを配置図で明示するとよい。

C.3 相互作用を表す図

通信図は、オブジェクト間のメッセージ交換を構造寄りに表す図である。シーケンス図が時間の流れを上から下へ示すのに対し、通信図はどのオブジェクトがどの相手とやり取りするかを強調する。オブジェクト数が少なく、関係の網目を見たい場合に向いている。

タイミング図は、時間の経過に伴う状態や値の変化を示す図である。リアルタイム性、タイムア

表 C.1 本文外で扱う UML 図種の用途

図種	主な対象	使う場面
オブジェクト図	具体的なインスタンス	多重度や関連の例示
パッケージ図	大きなまとまり	依存方向の整理
合成構造図	内部部品とポート	複雑な部品構成
コンポーネント図	実装単位	アーキテクチャ説明
配置図	実行環境	運用、セキュリティ、性能
通信図	オブジェクト間通信	相互作用の全体像
タイミング図	時間変化	時間制約の説明
相互作用概要図	処理の流れと相互作用	複数シナリオの俯瞰

ウト、センサ値、通信遅延など、時間制約が中心になるシステムで使う。業務アプリケーションでも、予約締切、キャンセル期限、通知の遅延を説明するときに役立つことがある。

相互作用概要図は、アクティビティ図に近い見た目で、複数の相互作用をまとめて示す図である。詳細なメッセージ列をすべて描く前に、代表的なシナリオの関係を俯瞰したいときに使う。

C.4 図を選ぶ基準

図を選ぶときは、先に読者の問いを一つに絞る。構造を知りたいならクラス図、部品境界を知りたいならコンポーネント図、実行環境を知りたいなら配置図、時間順の処理を知りたいならシーケンス図を選ぶ。一つの図で構造、時間、配置、責務をすべて説明しようとするとう読みにくくなる。

生成 AI に UML 図を作らせる場合も、同じ基準を使う。「予約作成のシーケンス図を PlantUML で作る」のように、図種と対象を限定する。出力後は、多重度、責務、メッセージ名、外部システム境界を人間が確認する。UML は設計の代替ではなく、設計を議論するための表記である [17]。

付録 D 用語集

本文で導入した主要用語を五十音順に示す。英字で始まる用語は末尾の英字の部にまとめた。各項目の末尾に主に扱う章を示す。

あ行

浅いコピー (shallow copy)

外側のコンテナだけを複製し、中のオブジェクトは共有するコピー。第 5 章。

アジャイル開発 (agile software development)

プロセスやツールよりも個人と対話を重視するアジャイルソフトウェア開発宣言に基づく開発思想の総称。第 8 章。

安全性 (safety)

故障や誤操作があっても人・財産・社会へ許容できない被害を与えない性質。第 2 章。

インスペクション (inspection)

役割を定め、事前準備を済ませ、欠陥発見を目的として進める最も厳格なレビュー形式。第 7 章。

受入基準 (acceptance criteria)

要求が満たされたと判断する条件。Given, When, Then の形で記述する。第 3 章。

埋め込み (embedding)

テキストなどをベクトルへ変換した表現。意味が近い文はベクトル空間でも近くなる。第 11 章。

ウォーターフォールモデル (waterfall model)

要求、設計、実装、テスト、運用を段階的に進める工程モデル。第 8 章。

ウォークスルー (walkthrough)

作成者が成果物を説明し、参加者が質問や指摘をするレビュー形式。第 7 章。

エクストリームプログラミング (Extreme Programming, XP)

テスト駆動開発、ペアプログラミング、リファクタリングなどの技術プラクティスで変化に対応する開発手法。第 8 章。

か行

回帰テスト (regression test)

変更によって既存機能が壊れていないことを確認するテスト。第 7 章。

開発プロセス (development process)

要求から運用までの作業をどの順序と反復で進めるかの取り決め。第 8 章。

可用性 (availability)

必要なときにシステムが利用可能である性質. 第 2 章.

型ヒント (type hint)

プログラムの意図を人間とツールへ伝える型注釈. Python の標準実行系は実行時に型を強制しない. 第 5 章.

カプセル化 (encapsulation)

データと操作をまとめ、内部表現を外部から隠すオブジェクト指向の基本概念. 第 4 章.

監査ログ (audit log)

指示、入力、ツール呼出し、結果を後から検証できるように記録する仕組み. 第 12 章.

関係データベース (Relational Database, RDB)

データを表の集合として管理し、一貫性を制約で守るデータベース. 第 6 章.

関係代数 (relational algebra)

選択、射影、結合などの演算で表を扱う理論. SQL の基盤. 第 6 章.

完了条件 (Definition of Done)

作業が完了したと観測できる条件を事前に明示する取り決め. 第 8 章.

機能要求 (functional requirement)

システムが何をするかを述べる要求. 第 3 章.

基本設計と詳細設計 (architectural and detailed design)

システム全体の構造を決める設計と、個々の部品の内部を決める設計. 第 4 章.

教師あり微調整 (Supervised Fine-Tuning, SFT)

人間が作った指示応答データで LLM の振る舞いを整える学習段階. 第 9 章.

共有渡し (call by sharing)

関数内の名前が渡されたオブジェクトを参照し、オブジェクトへの変更は呼出側にも見えるが再束縛は見えない Python の引数渡し. 第 5 章.

境界値分析 (boundary value analysis)

条件の境目に欠陥が集まりやすいことを利用するテスト技法. 第 7 章.

銀の弾丸 (silver bullet)

ソフトウェア開発の生産性を一気に何倍にもする単一技術は存在しないという Brooks の主張. 第 1 章.

偶発的複雑性 (accidental complexity)

道具や手法の未熟さに由来し、改善によって減らせる複雑さ. 第 1 章.

組合せ爆発 (combinatorial explosion)

条件の組合せがテストや検証で扱いきれない数に増える現象. 第 7 章.

検索拡張生成 (Retrieval-Augmented Generation, RAG)

外部文書を検索し、検索結果を文脈として与えて応答を生成する方法. 第 11 章.

検証マトリクス (verification matrix)

テスト、静的解析、レビュー、ログなどの証拠を要求へ対応付けて検証を管理する枠組み. 第 13

章.

さ行

事前学習 (pre-training)

大量のテキストから言語の統計的構造を学ぶ LLM の最初の学習段階. 第 9 章.

システム要求 (system requirement)

設計と実装へ接続できる形に整理された要求. 第 3 章.

主キーと外部キー (primary key, foreign key)

行を一意に識別する属性と, 別の表の主キーを参照して整合性を保つ属性. 第 6 章.

循環的複雑度 (cyclomatic complexity)

制御フローの分岐数に基づいてテストや保守の難しさを見積もる McCabe の指標. 第 2 章.

人間参加型制御 (Human-in-the-Loop, HITL)

AI の動作に人間が承認, 監督, 介入できるようにする設計. 第 12 章.

人間フィードバックによる強化学習 (Reinforcement Learning from Human Feedback, RLHF)

人間の選好を報酬として LLM の応答を調整する学習段階. 第 9 章.

信頼性 (reliability)

定められた条件で定められた期間, 故障せずに機能する性質. 第 2 章.

スクラム (Scrum)

経験主義に基づく開発の枠組み. 透明性, 検査, 適応の 3 つの柱を持つ. 第 8 章.

スパイラルモデル (spiral model)

リスク分析を中心に反復する工程モデル. 第 8 章.

正規化 (normalization)

更新異常を減らすために表を分解する設計法. 第 1 から第 3 正規形を扱う. 第 6 章.

生成 AI (generative AI)

文章, 画像, 音声, コードなどを新たに生成する人工知能の総称. 第 9 章.

制御語彙 (control vocabulary)

Issue, UML, テストファーストなど, AI に裁量を渡すために再利用する古典的ソフトウェア工学の語彙. 第 13 章.

説明責任 (accountability)

AI 出力を採用した開発者と組織が結果に責任を持つという原則. 第 14 章.

相互排他 (mutual exclusion)

共有資源を同時に使えるのは一つの主体だけであるという制御. ペトリネットで表現できる. 第 4 章.

ソフトウェア危機 (software crisis)

大規模開発が予算超過, 納期遅延, 品質欠如に陥る事態. 1968 年の NATO 会議で共有された. 第 1 章.

ソフトウェア工学 (Software Engineering, SE)

ソフトウェアの開発, 運用, 保守に対する体系的, 規則的, 定量的なアプローチの適用. 第 1 章.

た行

大規模言語モデル (Large Language Model, LLM)

大量のテキストで訓練され, 確率的に次のトークンを生成する言語モデル. 本書では確率的なソフトウェア部品として扱う. 第 9 章.

チャンキング (chunking)

検索対象文書の分割単位的设计. 小さすぎると文脈を失い, 大きすぎると精度を下げる. 第 11 章.

ツール権限 (tool permission)

エージェントのツールを読み取り, 提案, 書き込み, 外部送信, 不可逆操作に分けて最小権限にする設計原則. 第 12 章.

低ランク適応 (Low-Rank Adaptation, LoRA)

モデル全体を再学習せずに少数の追加パラメータでタスク適応する方法. 第 9 章.

データ独立性 (data independence)

プログラムをデータの物理的な保存方法から分離する性質. 第 6 章.

デザインパターン (design pattern)

生成, 構造, 振る舞いに関する再利用可能な設計の語彙. 第 4 章.

テスト駆動開発 (Test-Driven Development, TDD)

失敗するテストを先に書き, 最小実装で通し, リファクタリングする開発法. 第 7 章.

デッドロック (deadlock)

どの処理も先へ進めない状態. ペトリネットではどのトランジションも発火できないマーキングとして表れる. 第 4 章.

な行

名前とオブジェクトの束縛 (name binding)

Python の代入は箱に値を入れる操作ではなく, 名前をオブジェクトへ結び付ける操作であるという意味論. 第 5 章.

は行

ハイブリッド検索 (hybrid search)

ベクトル検索とキーワード検索を組み合わせ, 意味的類似と完全一致の両方を扱う検索方式. 第 11 章.

ハルシネーション (hallucination)

LLM が事実と異なる内容をもっともらしく生成する現象. 第 9 章.

非機能要求 (non-functional requirement)

性能, 信頼性, セキュリティなど, システムがどのようにあるべきかを述べる要求. 第 3 章.

品質モデル (quality model)

良いソフトウェアを機能適合性, 性能効率性, 信頼性などの観点に分解して扱う道具. ISO/IEC 25010 が代表. 第 2 章.

フルプルーフ (foolproof)

誤操作をそもそもできなくする設計. 第 2 章.

フェールセーフ (fail safe)

故障時にシステムを安全側へ移行させる設計. 第 2 章.

フェールソフト (fail soft)

故障時に機能を縮退させながら運転を継続する設計. 第 2 章.

フォールトトレラント (fault tolerant)

故障が起きても機能を継続する設計. 第 2 章.

深いコピー (deep copy)

入れ子のオブジェクトも再帰的に複製するコピー. 第 5 章.

ブラックボックステスト (black-box testing)

内部構造ではなく仕様からテストを設計する技法の総称. 第 7 章.

ブルックスの法則 (Brooks's law)

遅れているプロジェクトへの人員追加はさらに遅らせるという経験則. 第 1 章.

プロンプト (prompt)

タスク定義, 入力データ, 制約, 評価基準, 出力形式を含む LLM への小さな仕様. 第 10 章.

プロンプトインジェクション (prompt injection)

外部文書などに混入した悪意ある指示文がモデルの指示システムを乗っ取ろうとする攻撃. 第 11 章.

ペトリネット (Petri net)

並行性, 資源競合, 同期, デッドロックを表す数理モデル. プレース, トランジション, アーク, トークンからなる. 第 4 章, 付録 A.

ホワイトボックステスト (white-box testing)

内部構造を見てテストを設計する技法の総称. 命令網羅や分岐網羅を含む. 第 7 章.

本質的複雑性 (essential complexity)

解こうとする問題そのものに由来し, 取り除けない複雑さ. 第 1 章.

ま行

マイグレーション (migration)

表, 列, 制約, 索引の変更を版管理しながら適用する考え方. 第 6 章.

マーキング (marking)

ペトリネットである時点の各プレースのトークン分布。システムの現在状態を表す。第 4 章。

モデル (model)

目的に応じて重要な部分を選び、それ以外を捨てた対象の表現。第 4 章。

モデルカード (Model Card)

モデルの用途、性能、評価条件、制限、倫理的考慮を記録する文書形式。第 14 章。

や行

ユースケース (use case)

アクタが目的を達成するためのシステムとの相互作用の記述。第 3 章。

ユーザ要求 (user requirement)

利用者や顧客の言葉で述べられた要求。第 3 章。

ら行

リランキング (reranking)

初期検索で広く候補を集め、別モデルで関連度を再評価して並べ直す処理。第 11 章。

量子化 (quantization)

数値精度を落としてモデルの記憶容量と計算量を減らす操作。第 9 章。

レジリエンス (resilience)

障害から回復し機能を維持する能力。現代の分散システムの信頼性設計で用いられる。第 2 章。

英字

ACID (Atomicity, Consistency, Isolation, Durability)

トランザクションが満たすべき原子性、一貫性、分離性、永続性。第 6 章。

AI エージェント (AI agent)

LLM が状態を持ち、計画し、ツールを呼び、観測に応じて次の行動を選ぶ仕組み。権限を持つソフトウェア部品として設計する。第 12 章。

AI 駆動開発 (AI-driven development)

AI が候補生成やレビュー補助を担う前提で、どの裁量を AI に渡しどの判断を人間に残すかを設計する開発プロセス。第 13 章。

AIOps (AI for IT Operations)

運用ログ、メトリクス、アラートを AI で分析し障害対応を補助する考え方。第 14 章。

AI 利用ポリシー (AI usage policy)

AI の許可用途、禁止入力、検証義務、記録、責任の所在を定める文書。第 14 章。

Chain-of-Thought (CoT)

段階的な推論過程を出力させることで複雑な課題の正答率を高めるプロンプト手法. 第 10 章.

DevOps

開発と運用を対立させず, 継続的に価値を届けるための文化と実践. 第 8 章.

DORA 指標 (DORA metrics)

デプロイ頻度, 変更リードタイム, 変更失敗率, 復旧時間でデリバリ能力を測る指標群. 第 8 章.

EARS (Easy Approach to Requirements Syntax)

イベントや状態に応じて要求文の型を使い分ける構造化記法. 第 3 章.

ER モデル (Entity-Relationship model)

業務上の実体, 属性, 関連を表す概念データモデル. 第 6 章.

LLM-as-Judge

LLM に出力の評価を任せる補助手法. 評価者バイアスへの注意が必要. 第 10 章.

ReAct

推論と行動を交互に進めるエージェントの基本形. 第 12 章.

SOLID 原則 (SOLID principles)

単一責任, 開放閉鎖, リスコフ置換, インタフェース分離, 依存関係逆転からなる設計原則群. 第 4 章.

SRE (Site Reliability Engineering)

信頼性をソフトウェアエンジニアリングの問題として扱う考え方. SLO とエラーバジェットを含む. 第 8 章.

SWEBOK (Software Engineering Body of Knowledge)

ソフトウェア工学の知識の全体像を整理した体系. 第 1 章.

Transformer

注意機構を中心に据え, 語と語の関係を並列に捉えるニューラルネットワーク構造. LLM の基盤. 第 9 章.

UML (Unified Modeling Language)

ソフトウェア設計を図で表す標準記法. 第 4 章, 付録 C.

付録 E Python 基本構文

本付録は、第 8 章の `pytest` の説明と、第 5 章の Python プログラム読解へ進む前の確認用である。Python を初めて読む読者は、本付録の例を自分の環境で実行し、出力を説明できるようにしておく。本文では、ここで扱う初歩を前提として、参照、共有、例外、型ヒントへ進む。

E.1 環境と実行

Python では、仮想環境を使ってプロジェクトごとに依存関係を分ける。標準の `venv` を使う場合は、作業ディレクトリで仮想環境を作成し、有効化してからパッケージを入れる。 `uv` などの管理ツールを使う場合も、プロジェクト単位で環境を固定する考え方は同じである。

```
1 python --version
2 python hello.py
3 python -m pytest
```

Listing E.1 最小の実行確認

Python ファイルは、拡張子 `.py` を持つテキストファイルである。インデントが構文の一部であるため、タブと空白を混在させない。本書の例では、空白 4 文字のインデントに統一する。

E.2 変数、条件分岐、繰返し

変数は、値を入れる箱というより、オブジェクトへ付ける名前として理解する。整数、文字列、真偽値、リスト、辞書をまず読めれば、多くの例題に対応できる。条件分岐では `if`, `elif`, `else` を使う。繰返しでは、列挙できる対象に対して `for` を使い、条件が成り立つ間だけ続ける場合に `while` を使う。

```
1 items = ["book", "camera", "room"]
2
3 for item in items:
4     if item == "room":
5         print("reservation required")
6     else:
7         print("available:", item)
```

Listing E.2 条件分岐と繰返し

Python 3.10 以降では、`match` 文も使える。ただし、最初は `if` 文で十分である。制御構造を学ぶ目的は、短く書くことではなく、条件と分岐の意味を説明できるようにすることである。

E.3 関数とデータ構造

関数は、入力を受け取り、処理をまとめ、結果を返す単位である。戻り値は `return` で返す。処理を関数に分けると、テストしやすくなり、名前によって意図を説明できる。

```
1 def is_available(stock: dict[str, int], item: str) -> bool:
2     return stock.get(item, 0) > 0
3
4 stock = {"book": 2, "camera": 0}
5 print(is_available(stock, "book"))
6 print(is_available(stock, "camera"))
```

Listing E.3 関数と辞書の例

標準的なデータ構造として、リスト、タプル、辞書、集合がある。リストは順序付きの並び、タプルは変更しない組、辞書はキーと値の対応、集合は重複のない集まりである。どれを選ぶかは、「順序が必要か」、「変更するか」、「名前で取り出すか」、「重複を許すか」で判断する。

E.4 文字列、ファイル、例外

文字列の整形には f-string を使うと読みやすい。変数名を文字列の中に埋め込み、ログやメッセージを作れる。ファイルを読むときは、`with` 文を使う。処理が終わると自動的にファイルが閉じられるため、解放漏れを避けやすい。

```
1 from pathlib import Path
2
3 path = Path("items.txt")
4
5 try:
6     text = path.read_text(encoding="utf-8")
7     print(f"loaded {len(text)} characters")
8 except FileNotFoundError:
9     print(f"{path} is not found")
```

Listing E.4 ファイル読み込みと例外処理

例外は、通常の戻り値では扱いにくい異常を伝える仕組みである。何でも広く捕まえるのではなく、起こりうる例外を具体的に捕まえる。エラーを握りつぶすと、後の欠陥解析が難しくなる。

E.5 クラス、モジュール、pytest

クラスは、データと操作をまとめる仕組みである。最初から複雑な継承を使う必要はない。値をまとめるだけなら、`dataclass` が読みやすい。モジュールは Python ファイル単位の再利用単位であり、`import` で読み込む。

```
1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class Reservation:
5     item: str
6     user: str
7
8 def label(reservation: Reservation) -> str:
9     return f"{reservation.item}:{reservation.user}"
10
11 def test_label() -> None:
12     reservation = Reservation("book", "alice")
13     assert label(reservation) == "book:alice"
```

Listing E.5 dataclass と最小テスト

pytest では、test_ で始まる関数をテストとして実行する。まずは、入力、期待される出力、境界条件を一つずつ書く。第 7 章では、この最小形から、境界値、異常系、レビューへ広げる。

付録 F 画像生成系の概要

本付録は本文第 9 章でテキスト LLM に集中したため、画像生成系を必要とする読者向けに分離した。画像生成モデルは原理こそテキスト LLM と異なるが、第 10 章のプロンプト設計と評価、第 14 章の責任ある利用の議論は、そのまま画像生成にも適用できる。

F.1 生成モデルの考え方

画像生成モデルは、訓練データの画像集合が従う確率分布を近似し、その分布からの標本抽出として新しい画像を作る。テキスト LLM が次トークンの条件付き分布を学習するのに対し、画像生成では高次元の連続データの分布をどう表現するかが中心課題であり、その表現方法の違いが以降の各方式の違いになる。

F.2 拡散モデル

拡散モデル (diffusion model) は、画像に少しずつ雑音を加えて完全な雑音に至る前向き過程を定義し、その逆向きに雑音を除去する過程をニューラルネットワークで学習する [HJA20]。生成時は純粋な雑音から出発し、学習した除去過程を繰り返し適用して画像を得る。学習が安定しており生成品質が高い一方、生成に多数の反復を要するため計算コストが大きい。画素空間ではなく圧縮された潜在空間で拡散させる潜在拡散モデル (latent diffusion model) はこのコストを大幅に削減し、Stable Diffusion として広く利用された [Rom+22]。2020 年代半ば時点で画像生成の主流は拡散モデル系である。

F.3 GAN と VAE

敵対的生成ネットワーク (Generative Adversarial Network, GAN) は、画像を作る生成器と本物か偽物かを見分ける識別器を互いに競わせて訓練する枠組みである [Goo+14]。鮮明な画像を高速に生成できるが、訓練が不安定になりやすく、生成される画像の種類が訓練データの一部に偏るモード崩壊 (mode collapse) が起こりうる。

変分自己符号化器 (Variational Autoencoder, VAE) は、画像を低次元の潜在変数へ符号化し、そこから復号する過程を確率モデルとして学習する [KW14]。訓練は安定しており潜在空間の構造を解析しやすいが、単体では生成画像がぼやけやすい。現在では VAE は単独の生成器としてよりも、潜在拡散モデルの圧縮器・復元器としてなど、他方式の構成要素として使われることが多い。

表 F.1 に三方式の比較を示す。

F.4 マルチモーダル

テキストから画像を生成するには、言語と画像を結び付ける表現が必要である。CLIP (Contrastive Language-Image Pre-training) は大量の画像とキャプションの組から、対応する組が近くなるような共通の埋め込み空間を対照学習で獲得した [Rad+21]。DALL-E, Imagen, Stable Diffusion などのテキスト条件付き画像生成は、この種の言語・画像対応表現で拡散モデルなどの生成過程を条件付けることで実現されている。プロンプトの語彙選択が生成結果を大きく変える点はテキスト LLM と同様であり、第 10 章の系統的な比較評価の方法が有効である。

F.5 動画生成の概観

動画生成は、画像生成に時間方向の一貫性という制約を加えた問題である。フレーム間で物体の形状や運動が滑らかにつながる必要があり、時空間を扱う拡散モデルの拡張として研究が進んでいる。計算コストは静止画より桁違いに大きく、生成できる長さや解像度には制約が残る。個別のサービス名や到達水準は変化が速いため、本書では原理の水準に留める。

F.6 ソフトウェア工学から見た注意点

画像生成をソフトウェアに組み込む場合の工学的な注意点は、テキスト LLM と共通する部分が多い。出力が確率的で再現性の管理が必要なこと、評価には人手評価と自動指標の併用が必要なこと、訓練データ由来の権利と偏りの問題が残ることである。とくに生成画像の権利関係と、実在人物・実在物の合成に関わる責任は、第 14 章の AI 利用ポリシー設計の対象に含めて扱うべきである。

表 F.1 画像生成の三方式の比較

方式	学習の安定性	生成の速さ	主な位置付け
拡散モデル	安定	遅い (反復生成)	現在の主流。潜在拡散で高速化
GAN	不安定 (モード崩壊)	速い (一回の順伝播)	高速生成や画像変換で利用が残る
VAE	安定	速い	単体ではぼやける。他方式の圧縮器として利用

参考文献

- [Bec+01] Kent Beck et al. *Manifesto for Agile Software Development*. 2001.
- [Bec02] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [Bec04] Kent Beck. *Extreme Programming Explained: Embrace Change*. 2nd ed. Addison-Wesley, 2004.
- [Bey+16] Betsy Beyer et al., eds. *Site Reliability Engineering*. O'Reilly, 2016.
- [Boe78] Barry W. Boehm. *Characteristics of Software Quality*. North-Holland, 1978.
- [Boe88] Barry W. Boehm. “A Spiral Model of Software Development and Enhancement”. In: *IEEE Computer* 21.5 (1988), pp. 61–72.
- [Bre00] Eric Brewer. “Towards Robust Distributed Systems”. In: *PODC*. keynote. 2000.
- [Bro75] Frederick P. Brooks Jr. *The Mythical Man-Month: Essays on Software Engineering*. 邦訳『人月の神話』. Addison-Wesley, 1975.
- [Bro87] Frederick P. Brooks Jr. “No Silver Bullet — Essence and Accidents of Software Engineering”. In: *IEEE Computer* 20.4 (1987), pp. 10–19.
- [Bro+20] Tom Brown et al. “Language Models are Few-Shot Learners”. In: *NeurIPS*. 2020.
- [Che76] Peter Pin-Shan Chen. “The Entity-Relationship Model — Toward a Unified View of Data”. In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36.
- [Coc00] Alistair Cockburn. *Writing Effective Use Cases*. Addison-Wesley, 2000.
- [Cod70] Edgar F. Codd. “A Relational Model of Data for Large Shared Data Banks”. In: *Communications of the ACM* 13.6 (1970), pp. 377–387.
- [Dat03] C. J. Date. *An Introduction to Database Systems*. 8th ed. Addison-Wesley, 2003.
- [Dev+19] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *NAACL*. 2019.
- [Fag76] Michael E. Fagan. “Design and Code Inspections to Reduce Errors in Program Development”. In: *IBM Systems Journal* 15.3 (1976), pp. 182–211.
- [FHK18] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution, 2018.
- [Gam+94] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [GUW08] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. *Database Systems: The Complete Book*. 2nd ed. Pearson, 2008.
- [Geb+21] Timnit Gebru et al. “Datasheets for Datasets”. In: *Communications of the ACM* 64.12 (2021).

- [Goo+14] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. “Generative Adversarial Nets”. In: *NeurIPS*. 2014.
- [Gre+23] Kai Greshake et al. “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection”. In: *AISec*. 2023.
- [HJA20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *NeurIPS*. 2020.
- [Hu+22] Edward Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *ICLR*. 2022.
- [HF10] Jez Humble and David Farley. *Continuous Delivery*. Addison-Wesley, 2010.
- [IEE90] IEEE. *IEEE Std 610.12-1990: Standard Glossary of Software Engineering Terminology*. 1990.
- [11] *ISO/IEC 25010:2011 Systems and software engineering — SQuaRE — System and software quality models*. ISO/IEC. 2011.
- [23] *ISO/IEC 25010:2023 Systems and software engineering — SQuaRE — Product quality model*. ISO/IEC. 2023.
- [18] *ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering*. ISO/IEC/IEEE. 2018.
- [KW14] Diederik P. Kingma and Max Welling. “Auto-Encoding Variational Bayes”. In: *ICLR*. 2014.
- [Lev11] Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- [LT93] Nancy G. Leveson and Clark S. Turner. “An Investigation of the Therac-25 Accidents”. In: *IEEE Computer* 26.7 (1993), pp. 18–41.
- [Lew+20] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *NeurIPS*. 2020.
- [MY20] Yury A. Malkov and Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020).
- [Mar02] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2002.
- [Mav+09] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. “Easy Approach to Requirements Syntax (EARS)”. In: *Requirements Engineering Conference (RE)*. 2009.
- [McC76] Thomas J. McCabe. “A Complexity Measure”. In: *IEEE Transactions on Software Engineering* SE-2.4 (1976), pp. 308–320.
- [Mit+19] Margaret Mitchell et al. “Model Cards for Model Reporting”. In: *FAccT*. 2019.
- [Mur89] Tadao Murata. “Petri Nets: Properties, Analysis and Applications”. In: *Proceedings of the IEEE* 77.4 (1989), pp. 541–580.

- [NR69] “Software Engineering: Report of a Conference Sponsored by the NATO Science Committee”. In: *NATO Science Committee Conference*. Ed. by Peter Naur and Brian Randell. 歴史的なソフトウェア工学宣言の場. Garmisch, Germany, 1969.
- [17] *OMG Unified Modeling Language Specification, Version 2.5.1*. Object Management Group. 2017.
- [Ouy+22] Long Ouyang et al. “Training Language Models to Follow Instructions with Human Feedback (InstructGPT)”. In: *NeurIPS*. 2022.
- [Pat+21] David Patterson et al. *Carbon Emissions and Large Neural Network Training*. 2021. arXiv: [2104.10350](#).
- [Pet62] Carl Adam Petri. “Kommunikation mit Automaten”. PhD thesis. Technische Hochschule Darmstadt, 1962.
- [Rad+21] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. “Learning Transferable Visual Models from Natural Language Supervision”. In: *ICML*. 2021.
- [Raf+23] Rafael Rafailov et al. “Direct Preference Optimization: Your Language Model is Secretly a Reward Model (DPO)”. In: *NeurIPS*. 2023.
- [24] *Regulation (EU) 2024/1689 of the European Parliament and of the Council (Artificial Intelligence Act)*. European Union. 2024.
- [RG19] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *EMNLP*. 2019.
- [Rom+22] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. “High-Resolution Image Synthesis with Latent Diffusion Models”. In: *CVPR*. 2022.
- [RLL14] Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa. *PEP 484 — Type Hints*. 2014.
- [Roy70] Winston W. Royce. “Managing the Development of Large Software Systems”. In: *IEEE WESCON*. 1970.
- [Sch+23] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *NeurIPS*. 2023.
- [SS20] Ken Schwaber and Jeff Sutherland. *The Scrum Guide*. 版改訂あり. 2020.
- [Shi+23] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *NeurIPS*. 2023.
- [Sla19] Brett Slatkin. *Effective Python*. 2nd ed. Addison-Wesley, 2019.
- [SGM19] Emma Strubell, Ananya Ganesh, and Andrew McCallum. “Energy and Policy Considerations for Deep Learning in NLP”. In: *ACL*. 2019.
- [Vas+17] Ashish Vaswani et al. “Attention Is All You Need”. In: *NeurIPS*. 2017.
- [Wan+23] Xuezhi Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models”. In: *ICLR*. 2023.

- [WO24] Hironori Washizaki and Joanna Isabelle Olszewska, eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK), Version 4.0*. IEEE Computer Society, 2024.
- [Wei+22] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *NeurIPS*. 2022.
- [Yao+23] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *ICLR*. 2023.

索引

AI 駆動開発, 59

temperature, 42

top-k, 43

top-p, 43

Transformer, 42

インスペクション, 33

ウォークスルー, 33

グレースフルデグラデーション, 9

サーキットブレーカ, 9

システムテスト, 31

システム要求, 13

ソフトウェア危機, 4

ソフトウェア工学, ix

タイムアウト, 9

チャンキング, 52

デッドロック, 21

データシート, 71

データ独立性, 27

トランザクション, 30

ハイブリッド検索, 53

ハルシネーション, 43

パスアラウンド, 33

フォールバック, 9

マーキング, 20

モデル, 18

モデルカード, 71

ユーザ要求, 13

ユースケース, 14

リトライとバックオフ, 9

レジリエンス, 9

主キー, 29

事前学習, 42

体系的, 4

信頼性, 9

偶有的複雑性, 4

共有渡し, 24

単体テスト, 31

受入テスト, 31

可用性, 9

品質, 7

品質モデル, 7

回帰テスト, 31

埋め込み, 52

外部キー, 29

安全性, 9

定量的, 4

本質的複雑性, 4

機能要求, 13

浅いコピー, 24

深いコピー, 24

発火可能, 21

相互排他, 21

第 1 正規形, 29

第 2 正規形, 29

第 3 正規形, 29

結合テスト, 31

規則的, 4

量子化, 44

非機能要求, 13